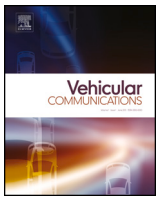
Vehicular Communications 53 (2025) 100900



Contents lists available at ScienceDirect

Vehicular Communications

journal homepage: www.elsevier.com/locate/vehcom





Task offloading and multi-cache placement based on DRL in UAV-assisted MEC networks ☆

Kai Xue, Linbo Zhai *, Yumei Li, Zekun Lu, Wenjie Zhou

School of Information Science and Engineering, Shandong Normal University, Jinan, Shandong, China

ARTICLE INFO

Keywords:
UAV
Mobile edge computing
Task offloading
Multi-cache placement
Deep reinforcement learning

ABSTRACT

Unmanned aerial vehicles (UAVs) are being developed as a promising technology to assist mobile edge computing (MEC) systems due to their reliable wireless communication, flexible computing service capabilities, and flexible deployment. However, in the face of huge information and demanding task delay, it is a challenging problem to reduce the system cost. This paper studies task offloading and cache space placement for ground users, and proposes a multi-UAV assisted computing framework, which is a four-layer transmission system composed of ground users (UE), UAVs, edge data centers (EDC) and remote clouds. By jointly optimizing UAV cache space, flight path, offloading decision, channel ratio, and battery power, we formulate the problem to minimize the long-term average weighted cost of the system under the constraint of cache space and computing resources. Since this problem is a mixed integer variable problem, we design a task offloading and cache placement algorithm based on deep reinforcement learning, namely the Cooperative Long-term Average Cost Minimization Optimization Algorithm (CLACMO). Firstly, we transform the mixed action variable space by using embedded tables and conditional variational autoencoders (VAE) combined with latent space, and map the mixed action variable to the latent action space. This approach effectively unifies discrete and continuous actions, addressing the challenge of mixed action spaces that traditional deep reinforcement learning algorithms struggle with. Secondly, based on the deep reinforcement learning (DRL), we achieve the long-term system average weighted cost minimization more efficiently under the constraints of task offloading and cache placement. The results show that compared with the PER-UOS-RL, MASAC, and MADDPG algorithms, the average reward has increased by 54.5%, 66.7%, and 69.7% respectively, and the average task completion rate has increased by 12.9%, 38.1%, and 9.11% respectively, demonstrating the effectiveness of our novel method.

1. Introduction

With the rapid development of Internet of Things (IoT) technology [11][43], a large number of promising applications and services have promoted the realization of intelligent environment [49][12], including intelligent transportation, virtual reality, smart city, etc., which has greatly promoted the progress and development of intelligent applications [9], [28]. These applications are typically compute-intensive and latency-sensitive, and require massive amounts of low-power and size-constrained mobile devices to perform tasks. However, ground user devices (UEs) are often equipped with limited computing resources and limited battery life, which will limit their effectiveness in the corresponding scenarios. Mobile edge computing (MEC) will provide effective solutions to the above problems [19].

Mobile Edge Computing (MEC) emerges as a crucial solution to these challenges. By moving computing resources closer to the edge of the network, MEC allows UEs to offload their computing tasks to edge servers for execution. In contrast to traditional cloud computing, where data

☆ This work was supported in part by National Key R&D Program of China under Grant 2023YFE0208800, in part by the Key Research and Development Program of Shandong Province, China, under Grant 2017GGX10142, in part by the National Natural Science Foundation of China under Grant 62301301 and in part by Shandong Provincial Natural Science Foundation, China, under Grant ZR2022MF354, ZR2023MF025 and ZR2023MF090.

* Corresponding author.
E-mail addresses: 1635452179@qq.com (K. Xue), zhai@mail.sdu.edu.cn (L. Zhai), 2378669917@qq.com (Y. Li), 1185997268@qq.com (Z. Lu), 420696422@qq.com (W. Zhou).

https://doi.org/10.1016/j.vehcom.2025.100900
Received 11 September 2024; Received in revised form 28 January 2025; Accepted 22 February 2025
Available online 26 February 2025
2214-2096/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

needs to be transmitted over long distances to centralized cloud servers for processing and then the results are sent back, MEC offers several distinct advantages.

Firstly, in terms of latency reduction, traditional cloud computing suffers from high latency due to the long transmission distance between UEs and cloud servers. For example, in real-time applications such as autonomous driving, where decisions need to be made within milliseconds, the delay incurred by transmitting data to the cloud and receiving the processed results back can be critical. MEC processes data at the network edge, significantly reducing the transmission distance and thus minimizing latency. This enables real-time processing of data, meeting the stringent latency requirements of time-sensitive applications [42].

Secondly, MEC improves computation offloading capabilities. Traditional cloud computing may not be able to handle the offloading demands of a large number of UEs efficiently, especially during peak usage periods. The centralized nature of cloud servers can lead to bottlenecks and resource contention. MEC distributes computing resources across multiple edge servers, providing more localized and dedicated processing power. This allows UEs to offload their tasks more effectively, reducing the computational burden on the UEs themselves and enhancing overall system performance [10].

Moreover, MEC contributes to optimizing energy efficiency. UEs often consume a significant amount of energy when transmitting data to distant cloud servers. By offloading tasks to nearby edge servers, MEC reduces the amount of data that needs to be transmitted over long distances, thereby saving energy for UEs. Additionally, edge servers can be designed to operate more energy-efficiently, further contributing to overall energy savings in the system.

With MEC, computing resources can be moved closer to the edge of the network [8], while UEs can offload their computing tasks to edge servers for execution. Recently, MEC has attracted a lot of attention in industry and science because it can greatly increase the computing power of mobile devices with energy saving and low cost. However, in some special scenarios, such as traffic jams, earthquake disasters, concert halls, network infrastructure, etc., may be damaged by natural disasters, or during peak hours, the communication resources and computing resources of edge servers are insufficient.

In order to deal with the problems in the above scenarios and ensure high quality of service (QoS), UAV-assisted mobile edge computing has become a promising scheme, which has aroused extensive research interest [45], [48]. Jeong et al. [14] considered a single UAV-assisted mobile edge computing system, but its purpose was to minimize energy consumption by jointly optimizing the offload ratio, bit allocation and UAV trajectory of upstream and downstream communication. In [34], maximizing energy efficiency schemes were considered by alternating approaches to solving two more manageable subproblems, such as optimal allocation of radio and computational resources for a given drone trajectory, and trajectory planning based on a given resource allocation scheme. Although the above research on UAV-assisted MEC system has achieved some results, the system delay problem is generally not considered. In the delay-related research, Xu et al. [38] focused on the computational delay of MEC system assisted by multiple UAVs with the goal of minimizing task completion time. Song et al. [31], studied the completion delay and energy consumption minimization of UAV multicast in a large-scale area. In addition, the optimal beam width and height are obtained by alternate iteration to minimize energy consumption. All the above articles focus on optimizing transmission delay and processing delay. However, in the task intensive scenario, a large number of tasks will be backlog in the resource pool, which means that the waiting delay is larger than the processing delay. Hence, it is important to optimize the waiting delay in this scenario.

In order to effectively shorten the service delay and reduce the energy consumption of computationally sensitive tasks, the current work mainly focuses on task offloading. Processing these tasks on edge computing servers requires the deployment of corresponding services, so some studies have studied the joint optimization of task offloading and service cache [27], [47], [3]. Authors of [14] studied a single unmanned aerial vehicle (UAV)-assisted mobile edge computing system, focusing is on reducing energy consumption by jointly optimizing the offloading ratio, bit allocation, and UAV trajectory. [34] optimized resource allocation and trajectory planning through an alternating method to improve energy efficiency. Their researches are difficult to be applied in practical scenarios that are extremely sensitive to delay, failing to meet the requirements of high-real-time tasks. Xu et al. [38] focused on the computational delay of a multi-UAV-assisted MEC system but ignored the optimization of waiting delay in task-intensive scenarios. Song et al. [31], studying the delay and energy consumption of UAV multicast, also did not optimize the waiting delay in task-intensive scenarios. This leads to the difficulty of effectively improving system performance when facing a large number of accumulated tasks in practical applications. Some researchers focused on the joint optimization, such as Bi et al. [23], [20], Zhang et al. [46], Yang et al. [39], and Ma et al. [24]. Bi et al. [23] solved the problems of resource allocation and service selection, but they did not consider the special factors of UAVs. When dealing with task offloading and cache placement, [20] ignored the complex interactions and resource limitations in the UAV - assisted scenario. The collaborative method proposed by Zhang et al. [46] did not fully consider the relationship between UAV flight and communication. When balancing delay and energy consumption, Yang et al. [39] did not conduct in - depth research on complex environments. After introducing satellites, Ma et al. [24] did not handle the related new problems well. All have problems to varying degrees in comprehensive optimization research. These deficiencies limit the wide application and effectiveness of their research findings in practice.

Therefore, there are still three challenges which are summarized as the following: (1) How to meet the diverse offloading requirements of users with the limited storage capacity of UAVs; (2) Hw to consider the limitations of UAV computing resources and the dependencies among tasks to achieve efficient task offloading and reduce the overall energy consumption of the system; (3) How to effectively reduce the total latency of user services by rationally utilizing edge servers.

In order to solve the above problems, this paper studies the task offloading related to edge computing and multi-cache placement. In order to further optimize the weighted sum of delay and energy consumption of the system, this paper takes into account the constraints of UAV cache space, UAV flight path, offloading decision, channel ratio, and UAV battery power. An efficient optimization framework for UAV-assisted MEC system is proposed. At the same time, in the details of the framework, this paper considers the bridge linkage relationship between the UAV and the edge data center (EDC), the dynamic channel allocation, and the size of the cached task data. Specifically, the main contributions of this paper are as follows:

(1) This paper designs a four-layer transmission system composed of ground users, UAVS, edge data centers and remote clouds. An average system cost for task offloading and multi-cache placement is developed to evaluate task completion delay and energy cost.
(2) By considering offloading decision variables, UAV flight path, EDC cache decision, channel bandwidth ratio variable, and UAV-EDC connection relationship, the problem is formulated to minimize long-term average system cost.
(3) In order to solve the problem, a cooperative long-term average cost minimization optimization algorithm is proposed. By introducing the embedded table of discrete action and the conditional variational autoencoder of continuous action, the representation space of mixed action is constructed on the basis of the autoencoder, and the mixed variable is transformed through the representation space. Based on the latent space,

a new Deep reinforcement learning (DRL) algorithm which can deal with the discrete and continuous mixed action space is proposed to solve the joint optimization problem.

The rest of the article is organized as follows. In Section 2, this paper reviews the relevant work. In Section 3, this paper introduces the system model. In Section 4, we propose a deep reinforcement learning algorithm based on joint embedded table-conditional variational autoencoder and latent space. In Section 5, the experimental simulation results are given. In Section 6, the full text is summarized.

2. Related work

With the emergence of delay-sensitive and resource-consuming user devices such as augmented reality and autonomous driving, this has greatly stimulated the need for low-latency access to computing resources. Mobile edge computing provides a new way to solve these challenges. Considering the necessity and non-ignorability of computing offload, the offloading of computing tasks has always been the central topic of edge computing. In the previous research work, different aspects of the task offloading problem are studied. In [16], the authors propose an efficient algorithm for calculating offloads to balance energy consumption. In [4], the author studies the scenario with dense distribution of edge nodes, and adopted stochastic game algorithm to solve the problem of resource shortage and high latency. In [24], the authors propose an improved queue-based multi-objective particle swarm optimization method to offload dependent tasks. In [29], the authors propose a new layered computing architecture by inserting an edge layer between the cloud layer and the traditional user layer. In [37], the authors analyze UAV-assisted dual-hop FSO/FSO systems with amplify-and-forward relaying, taking multiple factors into account. It obtains expressions for outage probability and average BER, investigates the impacts of parameters, and validates the findings through simulations. In [26], the authors comprehensively explore security threats in UAV communications, present secrecy metrics, review mitigation techniques, and discuss future research directions. In [32], the authors develop an opto-electronic system for NLOS communication in the visible wavelength range between UAV and the ground surface, conduct field experiments at wavelengths of 450 nm and 510 nm, analyze the symbol error rate and its standard deviation, and determine the maximum baseline distances for stable communication in different schemes.

In addition to the above traditional edge computing, the emerging UAV-assisted mobile edge computing has gradually entered people's field of vision. UAV assisted moving edge computing also faces some similar problems. For example, there is a sharp contradiction between the energy consumption of the UAV and the short delay of the system. In order to deal with these two kinds of conflicts, many researchers have conducted in-depth and extensive research on them in recent years. Yang et al. [39] and Zhang et al. [44] balance latency and energy consumption by introducing queues. Therefore, the Lyapunov based method is used to analyze the task queue, and the problem of energy consumption minimization is solved. Ma et al. [25] introduce the low Earth orbit (LEO) satellite to maximize the minimum average reachable rate among all IoT devices and the amount of data transmitted by IoT devices to Leo satellites, thus minimizing the energy consumption of UAV and avoiding the problem of data backlog of UAV. In addition, due to the limited onboard power of UAV, Zeng et al. [41] aim to maximize the weighted minimum of communication throughput of ground nodes through joint communication resource allocation and UAVs trajectory. By transforming this non-convex NP-hard problem into a partially observed Markov decision process, J et al. [5] propose a multi-agent depth deterministic strategy gradient algorithm to achieve the goal of minimizing the weighted sum of user average computing delay and UAV energy consumption. In [36], in order to ensure fairness among users, UAV is used as a relay to achieve a maximum QoE fairness by optimizing the flight direction, trajectory and flight distance of UAV. Asim M et al. [1] propose a variable population size trajectory planning and passive beamforming algorithm to minimize the weighted sum of system energy consumption and delay. Meanwhile, the research on multi-UAV working scenarios is as follows. In [15], the joint optimization problem of multi-UAV workload and task cache placement in complex scenarios and workload offloading and service cache hosting decision optimization problem are studied, and a specific request load offloading and service cache decision scheme based on hybrid deep reinforcement learning scheme is proposed. In [1], the joint optimization of UAV flight trajectory and intelligent reflector non-phase shift is studied to save energy consumption and task completion delay of multiple UAVs, and a variable population size trajectory planning and passive beamforming algorithm was proposed. In [7], the problem of UAV-assisted MEC system under the circumstance of limited computing resources in massive computing application scenarios is studied, and a new type of hybrid UAV-BS enabled MEC system is proposed. In [5], the problems of high cost, poor mobility and difficult emergency of large-scale deployment of MEC fixed edge computing nodes are studied, and a gradient algorithm based on multi-agent depth deterministic strategy is proposed.

At the same time, the edge service cache refers to the caching of application service and its related data in the edge server to facilitate the provision of services for users at the edge. Considering the resource limitation of edge server in computing and storage, service selection and resource allocation will become the key issues in designing service caching schemes. Bi et al. [2] apply MINLP (Mixed-Integer Nonlinear Programming) to describe the combinatorial optimization problem of resource allocation and service selection, and then solved the problem using closed-form expressions. In [20], under the constraints of MEC server computing resources and cache space, authors formulate task offloading and cache placement problems to maximize the hit rate and minimize the service delay. To solve this problem, a task offloading and cache placement (MOTOCP) algorithm based on multi-target artificial bee colonies is proposed. Similarly, Zhang et al. [46] propose a collaborative method, CCORAM, to solve the problem of co-optimization of resource allocation for service caching and multi-user computing offloading. In addition, in order to optimize both dynamic caching and computational offloading, Xu et al. [33] develop an online algorithm based on Lyapunov optimization and Gibbs sampling on the basis of fully considering the complexity of service request and service environment. In order to deal with the coordination problem of service caching and compute offloading, Tran et al. [23] introduce a cost model, which comprehensively considered the offloading cost from the three aspects of power consumption, service caching and cloud usage. The above study focuses on service while ignoring content caching. However, for the current technical requirements such as augmented reality, multiple users in the same neighborhood usually view the same scene, track the same environment, and need to identify the same objects, and content caching can effectively avoid task duplication and shorten service delay.

Different from previous studies, this paper studies the joint cache of service and compute content in UAV-assisted mobile edge computing system. This paper further studies the four-layer cache structure of users, UAVs, edge data centers and cloud server, and aims to minimize the long-term average cost. Based on this four-layer multi-content cache structure model, a Cooperative long-term average cost minimization optimization algorithm is proposed. The algorithm preprocesses the mixed action space by utilizing latent space and combining the embedded table and VAE coder. Then the algorithm optimizes the problem from two aspects of system delay and system energy consumption, and finally achieves better performance at a lower cost. Table 1 compares the proposed algorithm with the research in other related articles.

Table 1
Comparison of different studies in related works.

Reference	Single UAV	Multi UAV	MEC	cache placement	Offload decision	Reinforcement learning	Multi agent
[15]			✓		✓		
[4]			✓		✓		
[24]			✓	✓	✓		
[28]	✓		✓	✓	✓		
[38]	✓		✓	✓	✓		
[40]		✓		✓	✓		
[25]		✓	✓			✓	✓
[5]	✓						
[34]		✓	✓		✓		
[1]		✓	✓	✓	✓	✓	
[15]		✓	✓				
[7]			✓	✓	✓		
[2]			✓	✓	✓		
[44]	✓		✓		✓		
[37]	✓						
[26]		✓	✓			✓	✓
[32]	✓						
Our Proposed		✓	✓	✓	✓	✓	✓

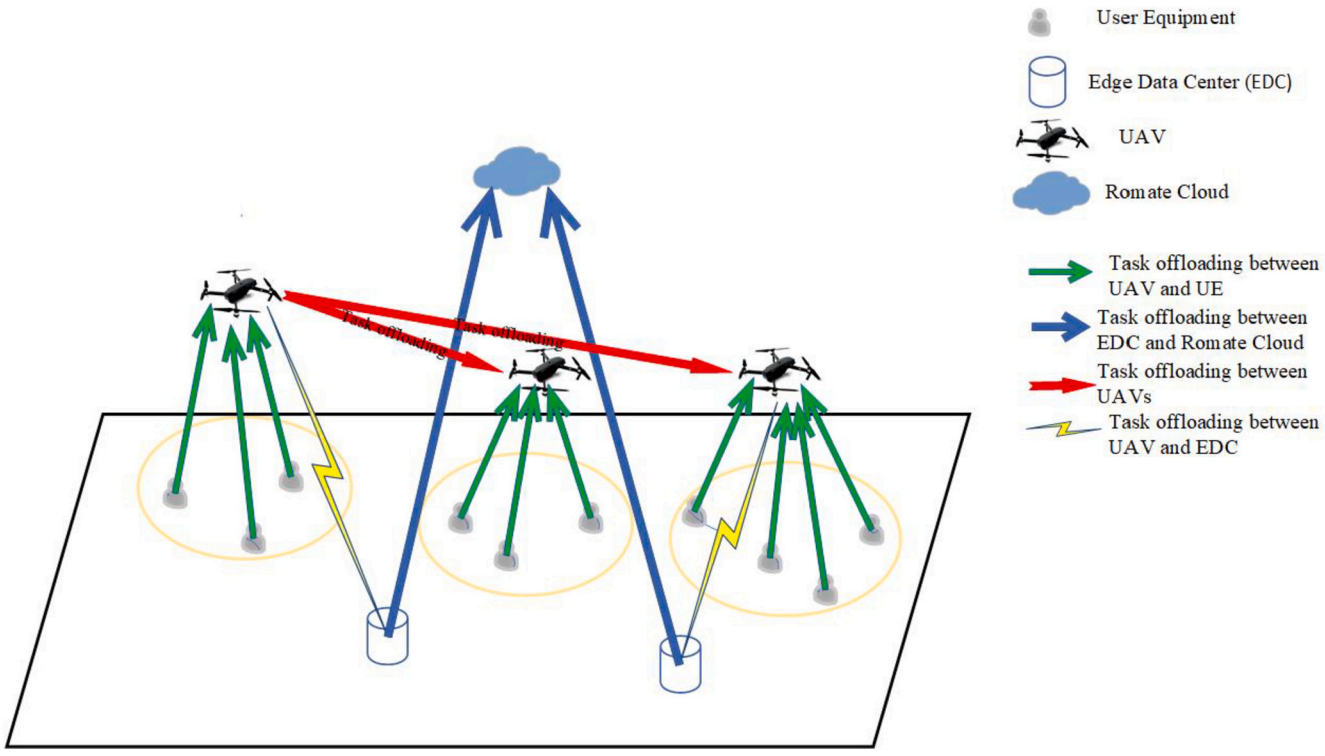


Fig. 1. The UAV-enabled MEC network.

3. System model

This paper deals with task offloading and multiple cache placement in moving edge computing, as shown in Fig. 1. We propose a network model as follows: a cloud server, M edge data centers (EDCs), J UAVs providing edge services and I users (UE). The set of ground users is defined as $\mathcal{J} \triangleq \{1, 2, \ldots, i, \ldots, I\}$. We set the total duration to T, and then the total duration can be evenly divided into N short time slots of slot length τ, satisfying $T = N \times \tau$, and having $t \in \mathbb{T} \triangleq \{1, 2, ..., N\}$. Given that τ is small enough, the position of the UAV will remain unchanged in slot t. UE has a task A_i, and each UE's task can be divided into several sub-tasks. The set of subtasks for each UE is represented by $\mathcal{R} \triangleq \{1, 2, ..., r, ..., R_i\}$. The r-th subtask of i-th UE is defined as $A_{i,r} = \{A_{i,r}^{size}, A_{i,r}^{cycle}, A_{i,r}^{duration}\}$, where $A_{i,r}^{size}$ is expressed as the data size of r-th subtask of i-th UE. $A_{i,r}^{cycle}$ indicates the number of CPU cycles required by i-th UE's r-th subtask to compute 1bit of UE's task data, and $A_{i,r}^{duration}$ indicates the maximum latency to complete the subtask. Edge data center (EDC) collection is $\mathcal{M} \triangleq \{1, 2, ..., m, ..., M\}$, the set of UAVs is $\mathcal{J} \triangleq \{1, 2, ..., j, ..., J\}$, the service cache set is defined as $\mathcal{D} = \{1, 2, ..., d, ..., D\}$, the set of content caches as $\mathcal{F} = \{1, 2, ..., f, ..., F\}$.

In Fig. 1, a green arrow labeled "Task offloading between UAV and UE" represents the task transfer between an Unmanned Aerial Vehicle (UAV) and a User Equipment (UE). A blue arrow labeled "Task offloading between EDC and Remote Cloud" shows the task transfer between an Edge Data Center (EDC) and a Remote Cloud. A red arrow labeled "Task offloading between UAVs" indicates the task transfer among UAVs. A lightning - bolt - shaped arrow labeled "Task offloading between UAV and EDC" depicts the task transfer between a UAV and an EDC.

We add an abbreviations table to enhance the readability of the manuscript. The abbreviations table used in this article are listed in Table 2.

Table 2
Abbreviations table.

Full name	Abbreviations
MEC	Mobile Edge Computing
UAV	Unmanned Aerial Vehicle
UE	User Equipment
EDC	Edge Data Center
CLACMO	Cooperative Long-term Average Cost Minimization Optimization Algorithm
SOADRL	Strategy Optimization Algorithm of Deep Reinforcement Learning

Table 3
Parameter Descriptions.

symbol	definition
$i, I, \mathcal{I}$	User index, user number, user set
$r, R, \mathcal{R}$	Subtask index, subtask number, subtask set
$m, M, \mathcal{M}$	EDC index, EDC number, EDC set
$j, J, \mathcal{J}$	UAV index, UAV number, UAV set
$d, D, \mathcal{D}$	service caches index, service caches number, service caches set
$f, F, \mathcal{F}$	computation content caches index, computation content caches number, computation content caches set
T, τ	Total duration of service, Slot length
$t, \mathbb{T}$	time slot label, short time slot collection
$\varrho_i, \varrho_j(t), \varrho_r$	UE coordinate, UAV coordinate, EDC coordinate
$\rho, \rho_{i,r}(t)$	Offloading decision set, offloading decision variable
$g_{i,r}(t)$,	Whether the subtask is offloaded to the UAV variable;
$h_{i,r}(t)$,	Whether the subtask is offloaded to the EDC variable;
$j_{i,r}(t)$,	Whether the subtask is offloaded to a remote cloud variable;
$l_{i,r}(t)$	Whether subtasks are offloaded to local variables;
ζ	Channel link classification variable
$P_{i,j}(t)$, $P_{j,m}(t)$,	UE transmission power; The UAV transmits power to the EDC;
$P_{j,j'}(t)$, $P_{m,Cloud}(t)$	The UAV transmits power to the UAV; The EDC transmits power to the remote cloud;
$S_m(t), \mathcal{S}$	EDC cache decision; EDC cache decision set
$C_j(t), \mathcal{C}$	UAV cache decision; UAV cache decision set
$s^{service}_{m,d,i,r}(t)$,	d-th service cache decision related to r-th subtask of i-th UE at m-th EDC;
$s^{content}_{m,f,i,r}(t)$	f-th computation content cache associated with r-th subtask of i-th UE at m-th EDC
$c^{service}_{j,d,i,r}(t)$,	d-th service cache decision related to r-th subtask of i-th UE at j-th UAV;
$c^{content}_{j,f,i,r}(t)$	f-th computation content cache associated with r-th subtask of i-th UE at j-th UAV
$S^{EDC}_m(t), S^{UAV}_j(t)$	Cache space size of m-th EDC, cache space size of j-th UAV

The UAV-assisted MEC system is based on a software-defined networking (SDN) framework that provides centralized network control and resource management. The cloud server collects prior real-time information of the system in the coverage area, including UE location, UAV location, UAV speed, data information, etc., which can be further distributed to each UE and UAV for information exchange. The main process of the task is described as follows. First, UE tasks will be divided into two parts: local computing and edge computing. Edge computing will first offload the task to the UAV, and then determine whether the specific location of the calculation is located in the UAV, EDC, or remote cloud server. Because some tasks of the edge server are difficult to process, which may lead to a higher task completion delay, some difficult tasks will be transferred to the remote cloud server for further analysis and execution.

The key symbols used in this article are listed in Table 3.

3.1. System model

Given that each time slot is small enough, each UAV can be considered stationary in each time slot. Thus, the trajectory of each UAV can be viewed as a sequence of discrete points throughout the mission execution cycle. The flight altitude of all UAVs is maintained at a fixed altitude H, so that the UAVs can avoid frequent ascent and descent to avoid obstacles. A 3-D Cartesian coordinate system is considered. The coordinates of the j-th UAV in the t-th timeslot are expressed as $\varrho_j(t) = \left(X_j(t), Y_j(t), H \right)$. Then the trajectory of j-th UAV can be expressed as $Q_j = \left\{ \varrho_j(1), ..., \varrho_j(t), ..., \varrho_j(N) \right\}$. Then, define $Q = \left\{ Q_1, ..., Q_j, ..., Q_J \right\}$ is defined as the set of trajectories of all UAVs. We define the r-th EDC coordinate as $\varrho_r = (X_r, Y_r, 0)$, and i-th UE is defined as the coordinates of $\varrho_i = (X_i, Y_i, 0), i \in \mathcal{I}$.

To enhance coverage, the UAV's trajectory should be within the target region, which is defined as $[0, x_{\max}] \times [0, y_{\max}]$. $\upsilon_j(t)$ is defined as the average speed of the UAV in slot t, υ_{max} is the maximum flight speed of the UAV. In the t-th time slot, to avoid signal interference and prevent collisions between UAVs, the minimum safe distance d_{min} should be maintained between any two UAVs. For example, the distance between j-th UAV and j'-th UAV should not be less than d_{min}. The definitions $\varrho_{Initial}$ and ϱ_{Final} represent the initial and end locations of the UAV, respectively. At the end of the service cycle, the UAV will return to its starting point. The initial position of the current time slot is the final position of the previous time slot, and the final position is the boundary point adjacent to the current coverage area and the next coverage area, this means that the UAV can be considered stationary for a time slot. Therefore, the following trajectory constraints are obtained:

$$\begin{cases} 0 \le X_j(t) \le x_{max}, \forall j \in \mathcal{J}, \forall t \in \mathbb{T} \\ 0 \le Y_j(t) \le y_{max}, \forall j \in \mathcal{J}, \forall t \in \mathbb{T} \\ v_j(t) = \frac{\left\| \varrho_j(t+1) - \varrho_j(t) \right\|}{\tau} \le v_{max}, \forall j \in \mathcal{J}, \forall t \in \left\{ \frac{\mathbb{T}}{N} \right\} \\ \left\| \varrho_{j'}(t) - \varrho_j(t) \right\| \ge d_{min}, \forall j \in \mathcal{J}, \forall j' \in \mathcal{J} \backslash \{j\}, \forall t \in \mathbb{T} \\ \varrho_j(1) = \varrho_{Initial}, \varrho_j(N) = \varrho_{Final}, \varrho_j(1) = \varrho_j(N) \end{cases} \tag{1}$$

3.2. Task offloading model

The offload decision variable defined is $\rho = \left\{ \rho_{i,r}(t) \middle| i \in \mathcal{I}, r \in \mathcal{R}, t \in \mathbb{T} \right\}$ to represent all the tasks collection of offloading decisions. The task offloading decision is described as follows:

$$\rho_{i,r}(t) = \langle g_{i,r,j}(t), h_{i,r,m}(t), j_{i,r,Cl}(t), l_{i,r,l}(t) \rangle. \tag{2}$$

$g_{i,r,j}(t)$ is the decision variable to determine whether the task will be offloaded on UAVs for processing. $g_{i,r,j}(t) \in \{0,1\}$ represents whether i-th UE will offload the r-th subtask on j-th UAV for execution. $g_{i,r,j}(t) = 1$ indicates that the i-th UE's r-th subtask is offloaded onto the j-th UAV for execution. $g_{i,r,j,j'}(t)$ is defined as the variable of whether i-th UE offloads r-th subtask to j'-th UAV by j-th UAV, then $g_{i,r,j,j'}(t) = 1$, conversely, $g_{i,r,j,j'}(t) = 0$. Define $g_{i,r,j,j'}(t)$ as the variable of whether i-th UE executes the r-th subtask on the j-th UAV. If it is executed on the j-th UAV, $g_{i,r,j,j'}(t) = 1$, and otherwise 0. Therefore construct $g_{i,r,j}(t) = 1 - g_{i,r,j,j'}(t) \times g_{i,r,j,j}(t)$. It should be noted that in the t-th time slot, when $j' = \mathrm{j}, j' \ne \mathrm{J} \backslash \{\mathrm{j}\}$, that is, in the same time slot, $g_{i,r,j,j}(t)$ and $g_{i,r,j,j'}(t)$ cannot be equal to 1 at the same time, which can ensure that whether it is offloaded on the current UAV or forwarded to other UAVs for further offloading. As long as the offloading is carried out on the UAV, it can be accurately judged that the mission is offloaded on the UAVs.

$g_{i,r,j}(t) \in \{0,1\}$ represents i-th UE whether to offload r-th subtask to j-th UAV for execution. $g_{i,r,j}(t) = 1$ indicates that the i-th UE's r-th subtask is offloaded onto the j-th UAV for execution. $h_{i,r,m}(t) \in \{0,1\}$ indicates whether i-th UE offloads r-th subtasks to m-th EDC for execution. $h_{i,r,m}(t) = 1$ indicates that the r-th subtask of i-th UE is offloaded to m-th EDC. $j_{i,r,Cl}(t) \in \{0,1\}$ represents i-th whether UE offload r-th subtasks to the remote cloud for execution. $j_{i,r,Cl}(t) = 1$ indicates that i-th UE's r-th subtask is offloaded to the remote cloud for execution. $l_{i,r,l}(t) \in \{0,1\}$ is defined as whether UE executes a subtask locally. $l_{i,r,l}(t)$ can be represented as $l_{i,r,l}(t) = 1 - \left(g_{i,r,j}(t) + h_{i,r,m}(t) + j_{i,r,Cl}(t) \right)$, and the existence $\left(g_{i,r,j}(t) + h_{i,r,m}(t) + j_{i,r,cl}(t) \right) \in \{0,1\}, (i \in \mathcal{I}, r \in \mathcal{R})$. $l_{i,r,l}(t) = 1$ indicates that the i-th UE's r-th subtask is executed locally. The offloading decision variables above indicate that the UE task is either offloaded at one of the UAV, EDC, remote cloud, or locally.

3.3. Communication model

The communication model is planned as follows: UE can choose to offload to the edge or perform task calculation locally. When UE chooses to offload to the edge, it needs to offload the task to the UAV first, and then determine whether there is calculation content cache $\mathcal{F}$ on the UAV and EDC. If there is calculation content cache $\mathcal{F}$, the UAV or EDC will directly return the calculation results to the UE. Otherwise, the task offloading request needs to check whether service cache $\mathcal{D}$ exists on the UAV and EDC to process the subtask. If service cache $\mathcal{D}$ exists on the UAV, the UAV executes the subtask. Otherwise, it will determine whether there is a service cache on the EDC, if so, the task request will be offloaded to the EDC, if the EDC still does not have a service cache $\mathcal{D}$, the task request is forwarded by the UAV to the EDC and then sent by the EDC to the remote cloud for calculation. $k_{j,m}(t)$ is defined as the connection variable between the UAV and the EDC. When the j-th UAV is connected to the m-th EDC, $k_{j,m}(t) = 1$, otherwise $k_{j,m}(t) = 0$. $k_{j,m}(t) = 1$ is determined based on the transmission rate size of all EDC within the communication range of the j-th UAV that have a computation-related subtask service cache or computational cache, and the UAV will select the EDC with which it has the highest transmission rate for communication connection. j-th UAV Due to the distance limitation of signal transmission, can only serve UE within its communication range R_j. In the t-th time slot, the distance between i-th UE and j-th UAV can be expressed as $d_{i,j}(t) = \sqrt{\left\| \varrho_i - \varrho_j(t) \right\|_2^2}$. The distance between the j-th UAV and the m-th EDC can be expressed as $d_{j,m}(t) = \sqrt{\left\| \varrho_m - \varrho_j(t) \right\|_2^2}$.

3.3.1. Communication model between UE and UAV

The wireless channel between the i-th UE and j-th UAV can be represented as a model combining LoS channel and NLoS channel, and the distance between the UAV and i-th UE is expressed as $d_{i,j}(t) = \sqrt{\left\| \varrho_i - \varrho_j(t) \right\|_2^2}$. The path gain is represented as follows:

$$h_{i,j}^{\zeta}(t) = \begin{cases} \frac{\left[\frac{4\pi d_{i,j}(t) \times f_r}{c} \right]^{-2}}{\eta_\zeta}, & \zeta = 0 \\ \frac{\left[\frac{4\pi d_{i,j}(t) \times f_r}{c} \right]^{-2}}{\eta_\zeta}, & \zeta = 1 \end{cases} \tag{3}$$

where $\zeta = 0$, and $\zeta = 1$ are defined as LoS and NLoS channel links, respectively, f_r is the carrier frequency and c is defined as the propagation rate of light. η_0 and η_1 represent the excess path loss in LoS and NLoS link free space path loss, respectively, which is determined by the environment (suburban, urban, tall buildings, or other scenarios). The probability of LoS channel link is $P_{i,j}^{LoS}(t) = \frac{1}{1 + Ae^{-B\left[\gamma_{i,j}(t) - A\right]}}$, in which A and B are constant value depends on the environment, and $\gamma_{i,j}(t) = 180/_{\pi} \sin^{-1}\left(H / d_{i,j}(t) \right)$ is defined as a UE elevation Angle. Then, the probability of NLoS channels defined as $P_{i,j}^{NLoS}(t) = 1 - P_{i,j}^{LoS}(t)$. Therefore, the average channel gain obtained is as follows:

$$h_{i,j}(t) = P_{i,j}^{LoS}(t) \times h_{i,j}^{0}(t) + P_{i,j}^{NLoS}(t) \times h_{i,j}^{1}(t). \tag{4}$$

The system works under the orthogonal frequency division multiple access (OFDMA), and the transmission rate between i-th UE and j-th UAV is defined as:

$$R_{i,j}(t) = g_{i,r,j}(t) \cdot \beta_{i,j}(t) \cdot W_{\mathcal{I}} \cdot \log_2\left(1 + \frac{P_{i,j}(t) \cdot h_{i,j}(t)}{\beta_{i,j}(t) \cdot W_{\mathcal{I}} \cdot N_0}\right) \tag{5}$$

where $P_{i,j}(t)$ represents the transmission power of i-th UE to UAV, N_0 is defined as the noise power spectral density, $\mathcal{I}$ is defined as the total channel bandwidth assigned to the UE transmission task, $\beta_{i,j}(t)$ represents the channel ratio, and $\beta_{i,j}(t) \in [0,1]$. According to the constraint of finite total bandwidth, this paper obtains:

$$\sum_{j \in \mathcal{J}} \sum_{i \in \mathcal{I}} \beta_{i,j}(t) \leq 1 \tag{6}$$

3.3.2. Communication model between the UAV and edge data center

The wireless channel between j-th UAV and m-th EDC can be characterized as a model combining LoS channel and NLoS channel, similar to the communication model above. Therefore, the following results can be obtained according to formulas (3), (4), (5) and the transmission rate between m-th EDC and j-th UAV is defined as:

$$R_{j,m}(t) = h_{i,r,m}(t) \cdot \beta_{j,m}(t) \cdot W_{\mathcal{J}} \cdot \log_2\left(1 + \frac{P_{j,m}(t) \cdot h_{j,m}(t)}{\beta_{j,m}(t) \cdot W_{\mathcal{J}} \cdot N_0}\right) \tag{7}$$

where $P_{j,m}(t)$ represents the transmission power of the j-th UAV to the m-th EDC, N_0 is defined as the noise power spectral density, $W_{\mathcal{M}}$ is defined as the total channel bandwidth assigned to the transmission task of the UAV to the EDC, $\beta_{j,m}(t)$ represents the channel ratio, and $\beta_{j,m}(t) \in [0,1]$. According to the constraint of finite total bandwidth, this paper obtains:

$$\sum_{j \in \mathcal{J}} \sum_{m \in \mathcal{M}} \beta_{j,m}(t) \leq 1 \tag{8}$$

3.3.3. Communication model between UAV and UAV

We define the max communication distance between UAVs as R_{UAV}. Any two UAVs can communicate with each other if the Euclidean distance between them does not exceed R_{UAV}. The distance between j'-th UAV and j-th UAV in t-th slot is $d_{j,j'}(t) = \sqrt{\left(X_j'(t) - X_j(t)\right)^2 + \left(Y_j'(t) - Y_j(t)\right)^2}$.

The rate of information transfer between UAVs can be expressed as:

$$R_{j,j'}(t) = g_{i,r,j,j'}(t) \cdot \beta_{j,j'}(t) \cdot W_{j,j'} \cdot \log_2\left(1 + \frac{P_{j,j'}(t) \cdot h_{j,j'}(t)}{\beta_{j,j'}(t) \cdot W_{\mathcal{J}} \cdot N_0}\right) \tag{9}$$

where $P_{j,j'}(t)$ represents the transmission power of the j-th UAV to the j'-th UAV, N_0 is defined as the noise power spectral density, $W_{\mathcal{J}}$ is defined as the total channel bandwidth assigned to the UAV to the UAV transmission task, and $\beta_{j,j'}(t) \in [0,1]$ represents the channel ratio. According to the constraint of finite total bandwidth, this paper obtains:

$$\sum_{j \in \mathcal{J}} \sum_{j' \in \mathcal{J}} \beta_{j,j'}(t) \leq 1 \tag{10}$$

3.3.4. Communication model between EDC and remote cloud

When there is no service cache and compute content cache on the EDC and UAV, the EDC offloads the task to the remote cloud server. The communication mode between EDC and cloud server is referred to in literature [21]. The transmission rate can be expressed as:

$$R_{m,Cloud}(t) = j_{i,r,Cl}(t) \cdot \beta_{m,Cloud}(t) \cdot W_{Cloud} \cdot \log_2\left(1 + \frac{P_{m,Cloud}(t) \cdot h_{m,Cloud}}{\beta_{m,Cloud}(t) \cdot W_{Cloud} \cdot N_0}\right) \tag{11}$$

where $P_{m,Cloud}(t)$ is defined as the power transmitted by the EDC to the remote cloud. $h_{m,Cloud}$ is defined as the channel gain between the EDC and the remote cloud. $\beta_{m,Cloud}(t) \times W_{Cloud}$ is defined as the bandwidth between the m-th EDC and the remote cloud, and W_{Cloud} is defined as the total bandwidth allocated for the EDC's transmission tasks to the remote cloud. $\beta_{m,Cloud}(t)$ represents the channel ratio and has $\beta_{m,Cloud}(t) \in [0,1]$. According to the constraint of finite total bandwidth, this paper obtains:

$$\sum_{m \in \mathcal{M}} \beta_{m,Cloud}(t) \leq 1. \tag{12}$$

3.4. Service and computation content cache model

Each UAV and EDC has different caching capabilities. This paper defines three kinds of cache decisions, $\mathcal{S} = \{S_m(t) | m \in \mathcal{M}\}$, and $\mathcal{Q} = \{Q_i(t) | i \in \mathcal{I}\}$ and $\mathcal{C} = \{C_j(t) | j \in \mathcal{J}\}$ representing the cache decision set of EDC, the cache decision set of UAV and the local cache decision set of UE. S_m and C_j are expressed as follows:

$$S_m(t) = \langle s_{m,d,i,r}^{service}(t), s_{m,f,i,r}^{content}(t) \rangle, \quad s_{m,d,i,r}^{service}(t), s_{m,f,i,r}^{content}(t) \in \{0,1\}, \tag{13}$$

$$c_j(t) = \langle c_{j,d,i,r}^{service}(t), c_{j,f,i,r}^{content}(t) \rangle, \quad c_{j,d,i,r}^{service}(t), c_{j,f,i,r}^{content}(t) \in \{0,1\}, \tag{14}$$

$$Q_i(t) = \langle q_{d,i,r}^{service}(t), q_{f,i,r}^{content}(t) \rangle, \quad q_{d,i,r}^{service}(t), q_{f,i,r}^{content}(t) \in \{0,1\}, \tag{15}$$

where $s^{service}_{m,d,i,r}(t)$ and $s^{content}_{m,f,i,r}(t)$ represent d-th service cache decision and f-th compute content cache related to r-th subtask of i-th UE at m-th EDC. When $s^{service}_{m,d,i,r}(t)$ and $s^{content}_{m,f,i,r}(t) = 1$, this indicates that the service cache and compute cache exist on the m-th EDC. Otherwise, the service cache and compute cache are not placed in m-th EDC. $c^{service}_{j,d,i,r}(t)$ and $c^{content}_{j,f,i,r}(t)$ represent d-th service cache decision and f-th compute content cache related to r-th subtask of i-th UE at j-th UAV, respectively. When $c^{service}_{j,d,i,r}(t)$ and $c^{content}_{j,f,i,r}(t) = 1$, this indicates that the service cache and compute cache exist on the j-th UAV. Otherwise, the service cache and compute cache are not placed in the j-th UAV. $q^{service}_{d,i,r}(t)$ and $q^{content}_{f,i,r}(t)$ represent d-th local service cache decisions and f-th local compute content cache related to r-th subtasks at i-th UE, respectively. When $q^{service}_{d,i,r}(t)$ and $q^{content}_{f,i,r}(t) = 1$, this indicates that the service cache and the compute cache exist locally. Otherwise, the service cache and compute cache are not placed locally.

This paper defines the cache space size of m-th EDC, cache space size of j-th UAV, and local cache space size of i-th UE as: $S^{EDC}_m(t)$, $S^{UAV}_j(t)$, and $S^{UE}_i(t)$, respectively. The space occupied by the service cache and compute content cache on the UAV and EDC cannot exceed the corresponding space size of both parties.

$$\sum_{i\in\mathcal{I},r\in\mathcal{R}} s^{service}_{m,d,i,r}(t)\cdot\theta_{i,r,d}(t) + s^{content}_{m,f,i,r}(t)\cdot o_{i,r,f}(t) \le S^{EDC}_m(t) \tag{16}$$

$$\sum_{i\in\mathcal{I},r\in\mathcal{R}} c^{service}_{j,d,i,r}(t)\cdot\theta_{i,r,d}(t) + c^{content}_{j,f,i,r}(t)\cdot o_{i,r,f}(t) \le S^{UAV}_j(t) \tag{17}$$

$$\sum_{i\in\mathcal{I},r\in\mathcal{R}} q^{service}_{d,i,r}(t)\cdot\theta_{i,r,d}(t) + q^{content}_{f,i,r}(t)\cdot o_{i,r,f}(t) \le S^{UE}_i(t) \tag{18}$$

where $\theta_{i,r,d}(t)$ and $o_{i,r,f}(t)$ represent the size of the service cache and the compute content cache, respectively.

3.5. Delay model

This paper assumes that each UE has a local task sequence and the task sequence of UAV and EDC, which follow the first-in first-out principle. When a new task arrives, the task is first stored in the UE's buffer, then the offload task will be moved out and stored in the buffer of the UAV, EDC, and some difficult tasks will be transmitted to the remote cloud for processing. Due to the superior storage and computing power of the remote cloud, there is no task waiting delay in the remote cloud.

3.5.1. UE task offloading model

$D^l_i(t)$ is defined as the data amount of local computing tasks, $D^{l,d}_i(t)$ is the local service cache computing tasks on i-th UE, $D^j_i(t)$ is defined as the data amount of tasks that i-th UE is offloaded to j-th UAV calculation, and $D^m_i(t)$ is defined as the data amount of tasks that i-th UE is offloaded to m-th EDC calculation. $D^{Cloud}_i(t)$ is defined as the data amount of tasks that i-th UE offloads to the remote cloud. The CPU cycle frequency of the i-th UE is constant $f^l_i(t)$, and its maximum frequency period is f^{max}_i. Therefore, the total CPU cycles that i-th UE can provide in one slot cycle is $f^l_i(t)\cdot\tau$. The subtask data amount of each UE is defined as $A^{size}_{i,r}$. The following constraints are then satisfied:

$$D^l_i(t) = \sum_{r=1}^{R_i} l_{i,r,l}(t)\times A^{size}_{i,r}(t) \tag{19}$$

$$D^{l,d}_i(t) = \sum_{r=1}^{R_i} q^{service}_{d,i,r}(t)\times l_{i,r,l}(t)\times A^{size}_{i,r}(t) \tag{20}$$

$$D^j_i(t) = \sum_{r=1}^{R_i} g_{i,r,j}(t)\times A^{size}_{i,r}(t) \tag{21}$$

$$D^m_i(t) = \sum_{r=1}^{R_i} h_{i,r,m}(t)\times A^{size}_{i,r}(t) \tag{22}$$

$$D^{Cloud}_i(t) = \sum_{r=1}^{R_i} j_{i,r,Cl}(t)\times A^{size}_{i,r}(t) \tag{23}$$

$f^j_i(t)$, $f^m_i(t)$, and $f^{Cloud}_i(t)$ are defined as the CPU frequency assigned by the j-th UAV to the i-th UE, the CPU frequency assigned by the m-th EDC to the i-th UE, and the CPU frequency assigned by the remote cloud to the i-th UE, respectively. The maximum CPU computing frequency of j-th UAV is defined as f^{max}_j, the maximum CPU computing frequency of m-th EDC is defined as f^{max}_j, and the maximum frequency assigned by the remote Cloud to i-th UE is defined as $F^{max}_{Cloud-i}$, Considering that the remote cloud has sufficient computing resources, each UE subtask can be assigned to a separate processor, with the following constraints:

$$\begin{cases} f^j_i(t) \ge 0, i\in\mathcal{I}, t\in\mathbb{T} \\ \sum_{i\in\mathcal{I}} f^j_i(t) \le f^{max}_j \end{cases} \tag{24}$$

Because the computing resources of the remote cloud server in this article are very rich, no other restrictions are made on it. $D^{j-j,d}_i(t)$, $D^{j-j',d}_i(t)$, $D^{j-m,d}_i(t)$ and $D^{j-Cloud}_i(t)$ are defined as the service cache tasks calculated on j-th UAV, the service cache tasks calculated on j'-th UAV, the service cache tasks calculated on m-th EDC and the remote cloud, respectively, which should meet the following constraints:

$$D^{j-j,d}_i(t) = \sum_{r=1}^{R_i} \left\{ c^{service}_{j,d,i,r}(t)\times g_{i,r,j,j}(t)\times A^{size}_{i,r}(t) \right\} \tag{25}$$

$$D_i^{j-j',d}(t) = \sum_{r=1}^{R_i} \left\{ c_{j,d,i,r}^{service}(t) \times g_{i,r,j,j'}(t) \times A_{i,r}^{size}(t) \right\} \tag{26}$$

$$D_i^{j-m,d}(t) = \sum_{r=1}^{R_i} \left\{ s_{m,d,i,r}^{service}(t) \times \left[h_{i,r,m}(t) \times A_{i,r}^{size}(t) + \left(1 - c_{j,d,i,r}^{service}(t) \cdot c_{j,f,i,r}^{content}(t) \right) \times g_{i,r,j}(t) \times A_{i,r}^{size}(t) \right] \right\} \tag{27}$$

$$D_i^{m-Cloud}(t) = D_i^{Cloud}(t) = \sum_{r=1}^{R_i} \left[j_{i,r,Cl}(t) \times A_{i,r}^{size}(t) + \left(1 - s_{m,d,i,r}^{service}(t) \cdot s_{m,f,i,r}^{content}(t) \right) \times h_{i,r,m}(t) \times A_{i,r}^{size}(t) \right] \tag{28}$$

$D_i^{j-j,f}(t)$, $D_i^{j-j',f}(t)$ and $D_i^{j-m,f}(t)$ are defined as the computational content cache tasks calculated on the j-th UAV, the computational content cache tasks calculated on the j'-th UAV and the computational content cache tasks calculated on the m-th EDC, so as to obtain:

$$D_i^{j-j,f}(t) = \sum_{r=1}^{R_i} \left\{ c_{j,f,i,r}^{content}(t) \times g_{i,r,j,j}(t) \times A_{i,r}^{size}(t) \right\} \tag{29}$$

$$D_i^{j-j',f}(t) = \sum_{r=1}^{R_i} \{ c_{j,f,i,r}^{content}(t) \times g_{i,r,j,j'}(t) \times A_{i,r}^{size}(t) \} \tag{30}$$

$$D_i^{j-m,f}(t) = \sum_{r=1}^{R_i} \left\{ s_{m,d,i,r}^{content}(t) \times \left[h_{i,r,m}(t) \times A_{i,r}^{size}(t) + \left(1 - c_{j,d,i,r}^{service}(t) \cdot c_{j,f,i,r}^{content}(t) \right) \times g_{i,r,j}(t) \times A_{i,r}^{size}(t) \right] \right\} \tag{31}$$

$D_i^{j-m}(t)$ is defined as the amount of processing tasks on the m-th EDC, and the maximum computing frequency of the CPU of the m-th EDC is defined as f_m^{max}. Therefore, you should meet the following constraints:

$$\begin{cases} f_i^m(t) \geq 0, i \in \mathcal{I}, t \in \mathbb{T} \\ \sum_{i \in \mathcal{J}} f_i^m(t) \leq f_m^{max} \end{cases} \tag{32}$$

In addition, this paper obtains the tasks of computing service cache on m-th EDC as follows:

$$D_i^{j-m}(t) = \sum_{r=1}^{R_i} \left\{ s_{m,d,i,r}^{service}(t) \left[\times h_{i,r,m}(t) \times A_{i,r}^{size}(t) + \left(1 - c_{j,d,i,r}^{service}(t) \cdot c_{j,f,i,r}^{content}(t) \right) \times g_{i,r,j}(t) \times A_{i,r}^{size}(t) \right] \right\} \tag{33}$$

3.5.2. Service request delay

The system delay includes service request delay, calculation delay and waiting delay. Considering that the size of the calculation result is much smaller than the size of the task data, the result return delay is ignored. Communication delays occur when UE subtasks are offloaded to the edge request cache or executed. This paper considers that the system works under OFDMA protocol, and the transmission rate between i-th UE and j-th UAV is defined as $R_{i,j}(t)$. Therefore, the request delay from UE to UAV can be expressed as:

$$T_{i,r,j}^{request}(t) = \frac{D_i^j(t)}{R_{i,j}(t)} \tag{34}$$

where $D_{i,j}(t)$ is defined as the amount of data that i-th UE transfers to the j-th UAV. When no content cache or service cache is computed on j-th UAV and a cache exists on j'-th UAV, the request is offloaded to j'-th UAV, and the transmission delay from j-th UAV to j'-th UAV is expressed as:

$$T_{i,r,j,j'}^{request}(t) = k_{j,m}(t) \cdot \frac{D_i^{j'}(t) + \left(D_i^j(t) - D_i^{j-j,d}(t) - D_i^{j-j,f}(t) \right)}{R_{j,j'}(t)} \tag{35}$$

where $R_{j,j'}(t)$ is defined as the task transfer rate from the j-th UAV to the j'-th UAV.

When no content cache or service cache is computed on the all UAVs and the EDC is cached, the request is offloaded to the EDC, and the transmission delay from the j-th UAV to the m-th EDC is expressed as:

$$T_{i,r,j,m}^{request}(t) = k_{j,m}(t) \cdot \frac{D_i^m(t) + \left(D_i^j(t) - D_i^{j-j',d}(t) - D_i^{j-j',f}(t) - D_i^{j-j,d}(t) - D_i^{j-j,f}(t) \right)}{R_{j,m}(t)} \tag{36}$$

where $R_{j,m}(t)$ is defined as the mission transfer rate from the j-th UAV to the m-th EDC.

When the computing content cache and service variable cache of the i-th UE's r-th subtask are not found in the EDC, the request is offloaded from the UAV to the remote cloud. The transmission delay from the UAV to the remote cloud is expressed as follows:

$$T_{i,r,m,Cloud}^{request}(t) = \frac{D_i^m(t) - D_i^{j-m,d}(t) - D_i^{j-m,f}(t)}{R_{m,Cloud}(t)} \tag{37}$$

Therefore, the total system service request delay in the t-th time slot can be expressed as:

$$T_{request}(t) = \sum_{i=1}^{I} \sum_{r=1}^{R_i} \sum_{j=1}^{J} \sum_{m=1}^{M} T_{i,r,j}^{request}(t) + T_{i,r,j,j'}^{request}(t) + T_{i,r,j,m}^{request}(t) + T_{i,r,m,Cloud}^{request}(t) \tag{38}$$

The total system service request delay can be expressed as:

$$T_{request} = \sum_{t=1}^{N} T_{request}(t) \tag{39}$$

3.5.3. Computation delay

The UE's tasks can be divided into sub-tasks based on its components, which can be executed locally or offloaded to the UAV, EDC, or remote cloud. Local execution: This article defines the CPU local computing power of i-th UE as $f_i^l(t)$, so the local computing delay is expressed as:

$$T_{i,r,local}^{comp}(t) = \frac{D_i^{l,d}(t) \cdot A_i^{cycle}}{f_i^l(t)} \tag{40}$$

UAV Execution: If there is a computational content cache $c_{j,f,i,r}^{content}(t) = 1$ associated with i-th UE's r-th subtask on j-th UAV, the subtask outputs the result directly without offloading or execution. Therefore, the execution delay is 0. When the subtask is offloaded on the UAV, if the UAV does not have the corresponding computational content cache $c_{j,f,i,r}^{content}(t) = 0$ but the service cache $c_{j,d,i,r}^{service}(t) = 1$, the task will be executed on the UAV. When there is neither a service cache nor a compute cache on the UAV but there is a compute cache or a service cache on another UAV, i-th UE's r-th subtask is forwarded from the j-th UAV to the j'-th UAV for computation. The CPU computing power allocated by the j-th UAV to the i-th UE is defined as $f_i^j(t)$. It should be noted that in order to avoid user resource requirements exceeding the maximum limit provided by the UAV, the sum of resources allocated to all UEs connected to the UAV must be less than or equal to the computing power of the UAV. So it should satisfy: $\sum_{i \in J} f_i^j(t) < f_j^{max}$. Therefore, the UAV calculation delay is expressed as:

$$T_{i,r,j,j}^{comp}(t) = \left(1 - c_{j,f,i,r}^{content}(t)\right) c_{j,d,i,r}^{service}(t) \frac{D_i^{j-j,d}(t) \cdot A_i^{cycle}}{f_i^j(t)} \tag{41}$$

$$T_{i,r,j,j'}^{comp}(t) = \left(1 - c_{j,f,i,r}^{content}(t)\right) c_{j',d,i,r}^{service}(t) \frac{D_i^{j-j',d}(t) \cdot A_i^{cycle}(t)}{f_i^j(t)} \tag{42}$$

where f_j^{max} is the maximum computing resource of j-th UAV.

EDC execution: When the requested UAV has neither content cache nor service cache, that is, $c_{j,f,i,r}^{content}(t) + c_{j,d,i,r}^{service}(t) = 0$, the subtask is offloaded to the EDC. If the calculation content cache $\left(s_{m,f,i,r}^{contect} = 1\right)$ of the r-th subtask of i-th UE exists on the m-th EDC, the calculation result of the subtask will be directly output, and the execution delay is 0. When the r-th subtask of i-th UE does not have a computational content cache $\left(s_{m,f,i,r}^{contect} = 0\right)$ on m-th EDC. If a service cache of i-th UE's r-th subtask $\left(s_{m,d,i,r}^{service} = 1\right)$ exists on the m-th EDC, the subtask will be executed on the EDC. The CPU computing power allocated by the m-th EDC to the i-th UE is defined as $f_i^m(t)$. It should be noted that in order to prevent the resource demand of the UE from exceeding the upper limit provided by the EDC, the sum of resources allocated to all UEs connected to the EDC must be less than or equal to the computing power of the edge data center. Therefore, it should satisfy: $\sum_{i \in \mathcal{I}} f_i^m(t) < f_m^{max}$. Therefore, the calculation delay of EDC is expressed as:

$$T_{i,r,m}^{comp}(t) = k_{j,m}(t) \cdot \left(1 - s_{m,f,i,r}^{content}(t)\right) s_{m,d,i,r}^{service}(t) \frac{D_i^{j-m,d}(t) \cdot A_i^{cycle}}{f_i^m(t)} \tag{43}$$

where f_m^{max} is defined as the maximum computing resource of m-th EDC.

Remote Cloud execution: When $c_{j,f,i,r}^{content}(t) + c_{j,d,i,r}^{service}(t) + s_{m,f,i,r}^{content}(t) + s_{m,d,i,r}^{service}(t) = 0$, compute content cache and service cache do not exist on the UAV and EDC. The task request will be offloaded to the remote cloud for execution. The remote cloud computing resource is defined as $F_{Cloud-i}^{max}$, and the CPU computing power assigned to i-th UE is defined as $f_i^{Cloud}(t)$. Therefore, the remote cloud latency is expressed as:

$$T_{i,r,Cloud}^{comp}(t) = \left(1 - c_{j,f,i,r}^{content}(t) - c_{j,d,i,r}^{service}(t) - s_{m,f,i,r}^{content}(t) - s_{m,d,i,r}^{service}(t)\right) \frac{D_i^{Cloud}(t) \cdot A_i^{cycle}}{f_i^{Cloud}(t)} \tag{44}$$

where $D_i^{Cloud}(t)$ is the number of tasks offloaded to the remote cloud.

Therefore, the calculation delay of the total system service in the t-th timeslot can be expressed as:

$$T_{comp}(t) = \sum_{i=1}^{I} \sum_{r=1}^{R_i} \sum_{j=1}^{J} \sum_{m=1}^{M} T_{i,local}^{comp}(t) + T_{i,r,j,j}^{comp}(t) + T_{i,r,j,j'}^{comp}(t) + T_{i,r,m}^{comp}(t) + T_{i,cloud}^{comp}(t) \tag{45}$$

Therefore, the total computation delay of system tasks can be expressed as:

$$T_{comp} = \sum_{t=1}^{N} T_{comp}(t) \tag{46}$$

3.5.4. Waiting delay

Local execution: For a subtask, the wait delay is the sum of all the locally executed subtasks in front of the subtask. The local wait delay of i-th UE's r-th subtask is the sum of the service delay of its previous $r-1$-th subtasks, the data amount of all locally executed subtasks before this subtask can be expressed as which can be expressed as:

$$D_{i,r}^{l,d}(t) = \sum_{r'=1}^{r-1} q_{d,i,r'}^{service}(t) \times l_{i,r',l}(t) \times A_{i,r'}^{size}(t) \tag{47}$$

where $A_{i,r'}^{size}(t)$ Indicates the data size of the r'-th subtask of the i-th UE.

The wait delay is calculated as the sum of all the locally executed subtasks before it, so the local wait delay is expressed as:

$$T_{i,r,local}^{wait}(t) = \sum_{r'=1}^{r-1} \frac{D_{i,r'}^{l,d}(t) \cdot A_i^{cycle}}{f_i^l(t)} \tag{48}$$

UAV execution: For a subtask, the wait delay is the sum of the computational delays from the start until the UAV executes the current subtask. The amount of data to be waited before the r-th subtask is calculated on this UAV and the amount of data to be waited before the r-th subtask is forwarded to other UAVs is expressed as:

$$D_{i,r}^{j-j,d}(t) = \sum_{r'=1}^{r-1} \left\{ c_{j,d,i,r'}^{service}(t) \times g_{i,r',j,j}(t) \times A_{i,r'}^{size}(t) \right\} \tag{49}$$

$$D_{i,r}^{j-j',d}(t) = \sum_{r'=1}^{r-1} \left\{ c_{j,d,i,r'}^{service}(t) \times g_{i,r',j,j'}(t) \times A_{i,r'}^{size}(t) \right\} \tag{50}$$

Therefore, the wait delay on this UAV and forwarding to other UAVs are denoted as:

$$T_{i,r,j,j}^{wait}(t) = \sum_{r'=1}^{r-1} \left(1 - c_{j,f,i,r'}^{content}(t)\right) c_{j,d,i,r'}^{service}(t) \frac{D_{i,r}^{j-j,d}(t) \cdot A_i^{cycle}}{f_i^j(t)} \tag{51}$$

$$T_{i,r,j,j'}^{wait}(t) = \sum_{r'=1}^{r-1} \left(1 - c_{j,f,i,r'}^{content}(t) \times c_{j,d,i,r'}^{service}(t)\right) c_{j',d,i,r'}^{service}(t) \frac{D_{i,r}^{j-j'd}(t) \cdot A_i^{cycle}}{f_i^j(t)} \tag{52}$$

EDC execution: For a subtask, the wait delay is calculated as the sum of the time delays of all the subtasks executed by EDC before it, the amount of data before the r-th subtask is expressed as:

$$D_{i,r}^{j-m,d}(t) = \sum_{r'=1}^{r-1} \left\{ s_{m,d,i,r'}^{service}(t) \times \left[h_{i,r',m}(t) \times A_{i,r'}^{size}(t) + \left(1 - c_{j,d,i,r'}^{service}(t) \cdot c_{j,f,i,r'}^{content}(t)\right) \times g_{i,r',j}(t) \times A_{i,r'}^{size}(t) \right] \right\} \tag{53}$$

so the EDC wait delay is expressed as:

$$T_{i,r,m}^{wait}(t) = \sum_{r'=1}^{r} k_{j,m}(t) \cdot \left(1 - s_{m,f,i,r'}^{content}(t)\right) s_{m,d,i,r'}^{service}(t) \frac{D_i^{j-m,d}(t) \cdot A_i^{cycle}}{f_i^m(t)} \tag{54}$$

Remote cloud execution: Considering the strong computing power of the cloud server, the waiting delay of its sub-tasks can be ignored.

Therefore, the total system service waiting time in the t-th slot can be expressed as:

$$T_{wait}(t) = \sum_{i=1}^{I} \sum_{r=1}^{R_i} \sum_{j=1}^{J} \sum_{m=1}^{M} T_{i,r,local}^{wait}(t) + T_{i,r,j,j}^{wait}(t) + T_{i,r,j,j'}^{wait}(t) + T_{i,r,m}^{wait}(t) \tag{55}$$

Therefore, the total waiting time of a system task can be expressed as:

$$T_{wait} = \sum_{t=1}^{N} T_{wait}(t) \tag{56}$$

Since each UE task has a maximum task completion delay threshold $A_i^{duration}(t)$, the i-th UE task completion time should meet the following requirements:

$$\begin{aligned} &\sum_{r=1}^{R_i} \left(T_{i,r,j}^{request}(t) + T_{i,r,j,m}^{request}(t) + T_{i,rm,Cloud}^{request}(t)\right) \\ &+ \left(T_{i,r,local}^{comp}(t) + T_{i,r,j}^{comp} + T_{i,r,m}^{comp}(t) + T_{i,r,Cloud}^{comp}(t)\right) + \left(T_{i,r,local}^{wait}(t) + T_{i,r,j}^{wait}(t) + T_{i,r,m}^{wait}(t)\right) \le A_i^{duration}(t) \end{aligned} \tag{57}$$

Based on the above discussion, the total system time delay can be expressed as:

$$T_{total} = T_{request} + T_{comp} + T_{wait} \tag{58}$$

3.6. Energy consumption model

The energy consumption mainly includes three parts: calculation energy consumption, offloading transmission energy consumption and flight energy consumption. The total battery energy of the UAV is defined as $E_{remain}(t)$, and the remaining energy of each time slot of the UAV is defined as $E_{remain}(t)$. This paper will describe the energy consumption model in detail as follows.

3.6.1. Computing energy consumption

Dynamic voltage scaling (DVS) and dynamic Power management (DPM) strategies are adopted to extend battery life by adjusting CPU cycle frequency. i-th UE's r-th subtask compute energy on local, compute energy on UAV, compute energy on EDC, and compute energy on remote cloud. It can be expressed as follows:

$$\begin{cases} E_{i,r}^{local}(t) = l_{i,r,l}(t) \cdot \kappa_{local} \cdot \left[f_i^l(t)\right]^3 \cdot T_{i,r,local}^{comp}(t) \\ E_{i,r,j}^{UAV}(t) = g_{i,r,j}(t) \cdot \kappa_{UAV} \cdot \left[f_i^j(t)\right]^3 \cdot T_{i,r,j}^{comp}(t) \\ E_{i,r,m}^{EDC}(t) = h_{i,r,m}(t) \cdot \kappa_{EDC} \cdot [f_i^m(t)]^3 \cdot T_{i,r,m}^{comp}(t) \\ E_{i,r}^{Cloud}(t) = j_{i,r,Cl}(t) \cdot \kappa_{Cloud} \cdot \left[f_i^{Cloud}(t)\right]^3 \cdot T_{i,Cloud}^{comp}(t) \end{cases} \tag{59}$$

where $\kappa_{local}, \kappa_{UAV}, \kappa_{EDC}$, and κ_{Cloud} represent the effective capacitance coefficient of the processing chip at the local, UAV, EDC, and remote cloud respectively.

Therefore, the total calculation energy consumption of the system in the t-th time slot can be expressed as:

$$E_{comp}(t) = \sum_{m=1}^{M}\sum_{j=1}^{J}\sum_{i=1}^{I}\sum_{r=1}^{R_i}\left(E_{i,r}^{local}(t) + E_{i,r,j}^{UAV}(t) + E_{i,r,m}^{EDC}(t) + E_{i,r}^{Cloud}(t)\right) \tag{60}$$

3.6.2. Offload transmission energy consumption

For offloading UE tasks, each UE and UAV is equipped with a communication module with variable transmission power. This paper defined $E_{i,j}^{local-UAV}(t)$ and $E_{i,j,m}^{UAV-EDC}(t)$ and $E_{i,m,Cloud}^{EDC-Cloud}(t)$ respectively as i-th UE of r-th subtasks offload to j-th UAV offload energy consumption, i-th Offloading energy consumption of UE's r-th subtask offloading on j-th UAV to m-th EDC, and offloading energy consumption of i-th UE's r-th subtask offloading on m-th EDC to remote cloud. It can be expressed as follows:

$$\begin{cases} E_{i,r,j}^{local-UAV}(t) = g_{i,r,j}(t) \cdot P_{i,j}(t) \cdot T_{i,r,j}^{request}(t) \\ E_{i,r,j,m}^{UAV-EDC}(t) = h_{i,r,m}(t) \cdot P_{j,m}(t) \cdot T_{i,r,j,m}^{request}(t) \\ E_{i,r,j,j'}^{UAV-UAV}(t) = g_{i,r,j,j}(t) \cdot P_{j,j'}(t) \cdot T_{i,r,j,j'}^{request}(t) \\ E_{i,r,m,Cloud}^{EDC-Cloud}(t) = j_{i,r,Cl}(t) \cdot P_{m,Cloud}(t) \cdot T_{i,r,m,Cloud}^{request}(t) \end{cases} \tag{61}$$

Based on [21], the $P_{m,Cloud}(t)$ can be expressed as $P_{m,Cloud}(t) = \frac{W_m(t)\cdot N_0}{h_{m,Cloud}} \cdot \left(2^{f_i^{Cloud}(t)} \Big/ _{W_m(t)\cdot\phi} - 1\right)$. Therefore, the total discharge transmission energy consumption of the system in the t-th timeslot can be expressed as:

$$E_{tran}(t) = \sum_{m=1}^{M}\sum_{j=1}^{J}\sum_{i=1}^{I}\sum_{r=1}^{R_i}\left(E_{i,r,j}^{local-UAV}(t) + E_{i,r,j,m}^{UAV-EDC}(t) + E_{i,r,j,j'}^{UAV-UAV}(t) + E_{i,r,m,Cloud}^{EDC-Cloud}(t)\right) \tag{62}$$

3.6.3. Flight energy consumption

In order to obtain the propulsion energy consumption of the UAV, the UAV is set to fly in a straight line at uniform speed within the time slot. A simplified model is considered in the existing work. Considering that the energy consumption is determined by the velocity vector, it can be expressed as:

$$P_j^{fly}(t) = P_0\left(1 + \frac{3V_j(t)^2}{U_{tip}^2}\right) + P_1\sqrt{\sqrt{1 + \frac{V_j(t)^4}{4V_0{}^4}} - \frac{V_j(t)^2}{2V_0{}^2}} + \frac{1}{2}d\rho s A_r V_j(t)^3 \tag{63}$$

$$E_j^{fly}(t) = P_j^{fly}(t) \cdot \tau \tag{64}$$

$$E_{fly}(t) = \sum_{j=1}^{J} E_j^{fly}(t) \tag{65}$$

The total energy consumption of the UAV cannot exceed the total energy of the battery $E_{battery}$, that is,

$$E_{battery} \geq \sum_{t=1}^{N}\sum_{i=1}^{I}\sum_{r=1}^{R_i}\sum_{m=1}^{M}\left(E_{i,r,j}^{UAV}(t) + E_{i,r,j,m}^{UAV-EDC}(t) + E_{i,r,j,j'}^{UAV-UAV}(t) + E_{fly}(t)\right) \tag{66}$$

Taking into account latency and energy consumption, the cost of the system is defined as:

$$Cost = \gamma_1 \cdot T_{total} + \gamma_2 \cdot E_{total} \tag{67}$$

$\gamma_1 \in [0,1]$ and $\gamma_2 \in [0,1]$ are defined as the weight coefficients of controlling delay and energy consumption respectively, and should satisfy $\gamma_1 + \gamma_2 = 1$.

3.7. Problem formulation

By jointly optimizing offloading decision variables $\rho = \left\{\rho_{i,r}(t)\middle| i \in \mathcal{I}, r \in \mathcal{R}, t \in \mathbb{T}\right\}$, UAV flight path set $Q = \left\{Q_1, ..., Q_j, ..., Q_J | j \in \mathcal{J}\right\}$, EDC cache decision variable $\mathcal{S} = \{S_m(t) | m \in \mathcal{M}, t \in \mathbb{T}\}$, UAV cache decision variable $\mathcal{C} = \left\{C_j(t)\middle| j \in \mathcal{J}, t \in \mathbb{T}\right\}$, channel collection ratio $B =$

$\{\beta_{i,j}(t), \beta_{j,m}(t), \beta_{j,j'}(t), \beta_{j,Cloud}(t) | i \in \mathcal{I}, j \in J, j' \in J \backslash \{j\}\}$, and the connection variable between the UAV and the EDC is $\mathcal{K} = \left\{ k_{j,m}(t) \middle| j \in \mathcal{J}, m \in \mathcal{M}, t \in \mathbb{T} \right\}$, we aim to minimize the long-term average system cost. The system cost function is the sum of weighted delay and energy consumption in the system. Hence, the joint optimization problem can be expressed as:

$$
\begin{aligned}
P_1 \quad & \min_{\rho,Q,S,C,\mathcal{B},\mathcal{K}} \left[\frac{1}{\xi} (\gamma^1 \cdot T_{total} + \gamma^2 \cdot E_{total}) \right] \\
s.t \quad & C_1 : \left(g_{i,r}(t) + h_{i,r}(t) + j_{i,r}(t) \right) \in \{0,1\}, \quad (i \in \mathcal{J}, r \in \mathcal{R}) \\
& C_2 : \sum_{i \in J} f_i^j(t) < F_j^{UAV}, \sum_{i \in J} f_i^m(t) < F_m^{EDC}, f_i^{Cloud}(t) \leq F_{Cloud-i}^{max} \\
& C_3 : \gamma_1 + \gamma_2 = 1 \\
& C_4 : \| \varrho_j(t) - \varrho_{j'}(t) \| > d_{min} \\
& C_5 : E_{battery} - \sum_{t=1}^{N} \sum_{i=1}^{I} \sum_{r=1}^{R_i} \sum_{m=1}^{M} \sum_{j'=1}^{J} \left(E_{i,r,j}^{UAV}(t) + E_{i,r,j,m}^{UAV-EDC}(t) + E_{i,r,j,j'}^{UAV-UAV}(t) + E_{fly}(t) \right) \geq 0 \\
& C_6 : (1)\ (6)\ (8)\ (10)\ (12)\ (13)\ (17)\ (18)\ (19)\ (57)
\end{aligned}
\tag{68}
$$

In P1, C1 ensures that UE tasks are offloaded either at one of the UAV, EDC, remote cloud, or locally. C2 indicates that the computing resources allocated by the UAV cannot exceed its maximum computing capacity, and the computing resources allocated by the EDC cannot exceed its maximum computing capacity. C3 indicates limits on delay and power consumption weight factors. C4 indicates that any two UAVs should maintain a minimum safe distance to prevent collision. C5 indicates that the total UAV energy consumption of the UAV should be less than the UAV battery power. Obviously, due to the non-convexity of the constraints, problem P1 is a non-convex function, and the complexity of the objective function makes it difficult to jointly optimize these variables. Moreover, considering the existence of continuous and discrete variables in the variables, it is difficult for traditional reinforcement learning algorithms to deal with such problem, so the CLACMO Method is designed in this paper. The details of the algorithm are shown in the next section.

Lemma 1. *P1 is NP-hard.*

Proof. We introduce the classic 0 - 1 knapsack problem for the reduction proof.

The 0 - 1 knapsack problem is defined as follows: Given a set of items, each item has its weight w_i and value v_i, and a capacity C of the knapsack. The goal is to select a set of items to put into the knapsack so that the total value of the items is maximized without exceeding the capacity of the knapsack. Its mathematical form is:

$$
\begin{aligned}
& \max \sum_{i=1}^{n} v_i x_i \\
& \text{s.t.} \sum_{i=1}^{n} w_i x_i \leq C \\
& \quad x_i \in \{0,1\}, i = 1, \cdots, n
\end{aligned}
$$

It is known that the 0 - 1 knapsack problem is NP - Hard.

In our problem $P1$, consider the following special case: Suppose there is $M = 1$ edge data center (EDC) and $J = 1$ unmanned aerial vehicle (UAV), and there is only one user ($I = 1$). The user has N tasks, and each task T_i has its computing resource requirement C_i (similar to the weight w_i in the 0 - 1 knapsack problem) and a value coefficient V_i related to the task (similar to the value v_i in the 0 - 1 knapsack problem).

We regard the cache space of the UAV and the EDC as the capacity C of the knapsack. For task T_i, it is offloaded to the UAV or the EDC for calculation, it is equivalent to selecting an item to put into the knapsack in the 0 - 1 knapsack problem. At this time, our goal is to minimize the long - term average weighted cost of the system. In this special case, it can be transformed into maximizing a function related to the task value (similar to maximizing the value in the 0 - 1 knapsack problem), while satisfying the constraints of cache space and computing resources (similar to the capacity constraint in the 0 - 1 knapsack problem).

Specifically, we define a new objective function $F = \sum_{i=1}^{N} V_i(1 - x_i)$, where x_i indicates whether task T_i is offloaded to the UAV or the EDC ($x_i = 0$ means offloading, $x_i = 1$ means not offloading). The constraint conditions are:

$$
\begin{aligned}
& \sum_{i=1}^{N} C_i x_i \leq C \\
& \quad x_i \in \{0,1\}, i = 1, \cdots, N
\end{aligned}
$$

It can be seen that in this special case, our problem $P1$ has a similar structure to the 0 - 1 knapsack problem. Since the 0 - 1 knapsack problem is NP - Hard and our problem $P1$ is a variant of the 0 - 1 knapsack problem in this special case, problem $P1$ is NP - Hard.

In the actual problem $P1$, due to the existence of multiple users, multiple UAVs and multiple EDCs, as well as the complex relationships between tasks, different cache decision variables, UAV flight path variables, channel ratio variables, etc., the problem becomes more complex, further demonstrating its NP - Hard nature. □

4. Algorithm design

In this section, we design a task offloading and cache placement algorithm based on deep reinforcement learning. First, the problem of minimum long-term average cost for UAV-assisted MEC systems can be viewed as a Markov decision process (MDP). Secondly, the unity of discrete action and

continuous action is realized by preprocessing mixed action variables based on latent space. Finally, based on the SOADRL algorithm to achieve the long-term system average weighted cost minimization more efficiently.

4.1. MDP construction

In a UAV-assisted MEC system, this paper optimizes offloading decisions, UAV position, EDC cache variables, UAV cache variables, computing resources, transmission power, channel ratio, and connection variables to minimize the long run average system cost. The status of the system in the next time slot depends only on the status and action of the current time slot. In this case, the problem of minimum long-term average cost for UAV-assisted MEC systems can be viewed as a Markov decision process (MDP). In the t-th slot, we observe the system state and then select the action. The system will generate a corresponding reward function to evaluate the action reflecting the current time slot. The goal is to maximize the long-term system reward function by utilizing optimization strategies that map state to action.

- State Space $\mathcal{S}$:
  If the channel quality of each transmission link is added to the state space, the state space will increase rapidly with the increase of the number of UAVs, thus increasing the complexity of the relevant algorithm. To reduce the size of the state space, the paper notes that the channel quality is dependent on the position of the UAV due to time-invariant signal interference between two locations in the model. In other words, the channel quality of the link will change with the position of the UAV. If the signal interference is constant, this paper can calculate the channel quality between the links according to the position of the UAV. Therefore, this article only adds the UAV position to the state space to describe the channel quality. In addition, this article no longer adds time-invariant variables to the state space. This article can use these variables directly during training without requiring them to be part of the state space. Therefore, in the optimization problem, the state space s(t) is composed of the UE calculation task and the three-dimensional coordinates of the UAV, expressed as:

$$s(t) = \{A(t), \boldsymbol{Q}(t)\} \tag{69}$$

  where $A(t) = \{A_1, A_2, ..., A_I\}$, $Q(t) = \left\{\varrho_1(t), \varrho_2(t), ..., \varrho_J(t)\right\}$ represent user task tuple and UAV location respectively. Considering that the dimension of calculating task features is dimension I, the position coordinate of UAV is dimension $3J$, and the total dimension of state space **s(t)** is $I+3J$, where I is the number of UEs, J is the number of UAVs, and M is the number of EDCs.
- Action Space $\mathcal{A}$:
  If the optimal offloading decision is directly defined as an action, the dimension of the action space is $IR \times (IRJ \times IRM \times IR \times IR)$. This increases the number of output neurons and leads to additional consideration of constraints, which increases the complexity of training. For each computing task of each UE, each subtask can be offloaded to UAV, or EDC, or remote cloud, or local computing, so there are $(J + M + 2)$ offloading schemes for each subtask, and $r \times (J + M + 2)$ offloading schemes for each UE task. To simplify discrete actions in action space, this paper uses $r_i(t) \in \{0, 1, ..., J, J + 1, ..., J + M, ..., J + M + 1\}$ represents the computing position of the sub-task of i-th UE, where $r_i(t) = 0$ represents local computing, $r_i(t) \in [J + 1, J + M]$ represents EDC computing, $r_i(t) \in [1, J]$ represents UAV computing, and the rest cases are remote cloud computing. In this way, the number of neurons in the task offloading variable can be effectively reduced. In addition to the task offloading variables, this paper must determine the UAV's mobility attributes, transmission power, computing resource allocation, and edge cache service decisions. Specifically, the action space in time slot t-th is defined as:

$$a(t) = \{\boldsymbol{r}(t), \triangle\boldsymbol{Q}(t), \boldsymbol{S}(t), \boldsymbol{C}(t), \boldsymbol{K}(t), \boldsymbol{B}(t)\} \tag{70}$$

  Among them, the $r(t) = \left[r_1(t), r_2(t), ..., r_{J+M+1}(t)\right]$ is the set of offloading tasks decisions in this paper.
  $\triangle\boldsymbol{Q}(t) = \left\{\triangle\varrho_1(t), \triangle\varrho_2(t), ... \triangle\varrho_J(t)\right\}$ is the mobility of UAVs. $S(t) = [S_1(t), S_2(t), ..., S_M(t)]$ indicates the EDC cache decision variable. $C(t) = \left\{C_1(t), C_2(t), ..., C_J(t) \middle| j \in \mathcal{J}\right\}$ indicates the UAV cache decision variable. $\mathcal{K}(t) = \left[k_{j,m}(t)\right]$ Indicates the connection variable relationship between the UAV and the EDC. $\mathcal{B} = \left\{\beta_{i,j}(t), \beta_{j,m}(t), \beta_{j,j'}(t), \beta_{j,Cloud}(t)\right\}$ The channel bandwidth ratio is $I \times J + J \times M + J \times (J - 1) + 1$ dimension. Action space dimension $a(t)$ is $3I + 2M + IJ + IM + MJ$, and continuous action space dimensions for $\mathcal{X}_p = J^2 + 2J(I + 1) + I(M + 1) + 2$.
- Reward Function: The objective of the formulaic task offloading optimization problem is to minimize the long run average system cost under certain constraints. Therefore, if an action yields a greater system benefit and satisfies all constraints, the action has a greater benefit. In addition, if certain constraints are not met, there is a corresponding penalty in the penalty function. The reward function is defined as follows:

$$Re(t) = \begin{cases} -\frac{1}{\xi}(\gamma^1 \cdot \psi^T \cdot T_{total} + \gamma^2 \cdot \psi^E \cdot E_{total}), \textit{if the constraints are met} \\ P_u, otherwise \end{cases} \tag{71}$$

  where P_u represents a positive number and P_u represents the penalty for an action that does not meet the constraint condition.
  Because the above MDP has a discrete continuous mixed action space, the traditional deep reinforcement learning algorithm is not suitable. If the mixed action space variables are directly converted into discrete or continuous action Spaces, this operation will cause degradation of model performance due to scalability problems and increased approximate complexity. In order to properly deal with this problem, this paper proposes a new algorithm based on mixed action representation [32].

4.2. Latent space

In the formulation of MDP, there are discrete variables $\{\boldsymbol{B}(t), r(t), \boldsymbol{S}(t), \boldsymbol{C}(t), \boldsymbol{K}(t)\}$ and continuous variable $\{A(t), \triangle Q(t)\}$. Hybrid action representation can transform the discrete-continuous hybrid action space problem into a continuous strategy learning problem considering the interdependence between two heterogeneous components. *p(t)* is used to uniformly refer to continuous actions, and *q(t)* is used to represent discrete actions. Based on [18][13], we introduce this method in detail from two aspects of perception-dependent mixed action coding and decoding.

There are $r \times (J + M + 2)$ possible offloading schemes for each user task, that is, there are $r \times (J + M + 2)$ positions for calculating offloading for each subtask. This article first to create a nested table $G_w \in \mathbb{R}^{r \times (J+M+2) \times l_1}$ be learning parameters to define the omega $r \times (J + M + 2)$, a discrete action. In the table, each row $g_{\omega,q(t)} = G_w\left(q(t)\right)$ is a continuous variable of the l_1 dimensional discrete action. It should be noted that there are I

users making decisions in each time slot, so I embedded tables need to be built for learning. However, the action space of each UE and the meaning of each action is the same, which means that all UEs can share a common embed table.

In order to construct an l_2 dimensional latent representation space of continuous parameters, a traditional Variational Auto Encoder (VAE) is used in [17]. In a mathematical formula, mixing action space is given $a(t) = \left(p(t), q(t)\right)$, and the state space $s(t)$, and the encoder $q_\phi\left(z(t)\middle|p(t), s(t), g_{\omega,q(t)}\right)$. Parameter ϕ maps $p(t)$ to the latent variable $z(t) \in \mathbb{R}^{l_2}$ conditioned on the state space $s(t)$ and $g_{\omega,q(t)}$.

Gaussian distribution $\Gamma(\mu_{q(t)}, \sigma_{q(t)})$ is used to describe the encoder $q_\phi\left(z(t)\middle|p(t), s(t), g_{\omega,q(t)}\right)$. The encoder will output the mean $\mu_{q(t)}$ and the standard deviation $\sigma_{q(t)}$ of the latent distribution. Sampling from this distribution, we obtain the latent characterization $z(t) \sim \Gamma\left(\mu_{q(t)}, \sigma_{q(t)}\right)$.

Under the same conditions, with parameters of bits of decoder $q_\psi\left(\tilde{p}(t)\middle|z(t), s(t), g_{\omega,q(t)}\right)$, reconstruct from $z(t)$ in the reconstruction of continuous parameter $\tilde{p}(t)$. Given a sample $\tilde{p}(t) = q_\psi\left(z(t), s(t), g_{\omega,q(t)}\right)$, decoder determines the decoding of the sample, and gets the $\tilde{p}(t) = q_\psi\left(z(t), s(t), g_{\omega,q(t)}\right)$. In addition, we can decode the discrete parameter $q(t)$ by looking for $g_{\omega,q(t)}$ in the embedded table through the adjacent query.

An encoder is used to construct a hybrid action representation space ($\in \mathbb{R}^{I \times l_1 + l_2}$). In addition, the latent variables $g \in \mathbb{R}^{I \times l_1}$ and $z(t) \in \mathbb{R}^{l_2}$ can be decoded into the mixing action space $\left(p(t), q(t)\right)$ based on decoders. For representation, the encoding and decoding processes are summarized as follows:

Encoding:

$$g_{\omega,q(t)} = G_w\left(q(t)\right), z \sim q_\phi\left(\cdot\middle|p(t), s(t), g_{\omega,q(t)}\right) \tag{72}$$

Decoding:

$$q(t) = argmin_{p'(t)\in p(t)}\left\|g_{\omega,p'(t)} - g\right\|_2, \tilde{p}(t) = q_\psi\left(z(t), s(t), g_{\omega,q(t)}\right) \tag{73}$$

The first term represents the square of the reconstruction error, and the second term represents the Kullback-Leibler divergence (DKL) between the variational posteriori of $z(t)$ and the standard Gaussian prior.

Since the mixed actions have different effects on the environment, this paper combines a cascade structure that follows the conversion network of conditional VAE decoding. Experience for any sample $\{s(t), a(t), s'(t)\}$, this paper defines state surplus for the $\delta_{s(t),s'(t)} = s'(t) - s(t)$. By introducing the cascade structure into the decoder, this paper generates predictions based on the following processes:

$$\bar{\delta}_{s(t),s'(t)} = q_\psi\left(z(t), s(t), g_{\omega,q(t)}\right), for z(t), s(t), g_{\omega,q(t)} \tag{74}$$

Then the square of the L_2 prediction error is:

$$L_D(\phi, \psi, \omega) = \mathbb{E}\left[\left\|\bar{\delta}_{s(t),s'(t)} - \delta_{s(t),s'(t)}\right\|_2^2\right] \tag{75}$$

Therefore, this paper minimizes the final training loss:

$$L_H(\phi, \psi, \omega) = L_V(\phi, \psi, \omega) + \alpha L_D(\phi, \psi, \omega) \tag{76}$$

where α is a weight parameter that depends on the importance of the loss associated with the dynamic prediction representation. The dimension of the state space of the system is **dim**, and the dimension of the continuous strategy $p(t)$ is χ_p.

Although the latent space of the mixed action representation increases the complexity of the algorithm, it is necessary. Therefore, it is more precise to use latent spaces that can be converted between continuous output values and discrete output variables. Considering that discrete variables and continuous variables are coupled to each other in the mixed space, we cannot just code the discrete variables, but we must code the entire mixed action space to ensure the correlation of the variables. The function of hybrid action space is to continuously process the discrete action of the initial hybrid action space through the coder as the means and the latent space as the carrier, so as to solve the problem of hybrid action space. Compared with the original space, the discrete action and continuous action after encoder and decoder processing can be unified in the latent space.

4.3. Strategy optimization algorithm of deep reinforcement learning (SOADRL) algorithm

SOADRL algorithm (Algorithm 1) is a deterministic strategy reinforcement learning algorithm to solve mixed variable space. Based on [22], its structure is actor-critic structure, in which the actor network adopts DDPG algorithm and the critic network adopts Dueling DQN algorithm. Actor networks map variables of various states to corresponding behaviors, thus influencing the decision-making process. The critic network evaluates the latent rewards associated with different actions in a given environment and influences the value of actions.

The actor network takes the environmental state $s(t)$ as input and produces a set of latent action vectors, represented by g and z, where $g, z = \pi_\zeta\left(s(t)\right), g \in \mathbb{R}^{Ml_1}, z \in \mathbb{R}^{l_2}$. Next, this paper uses a decoder to decode the latent action vector (g, z) into the corresponding discrete continuous mixed action $a(t) = \left(p(t), q(t)\right)$. The action vector g represents the discrete variable after $q(t)$ is decoded, and the action vector z represents the continuous variable after $p(t)$ is encoded. So the decision making process is divided into two parts. First, DDPG outputs continuous actions corresponding to all discrete actions based on the observed state. Then, Dueling-DQN can get the discrete action with the largest q value when it inputs the observed state and the obtained continuous action. In this way, discrete optimal behavior and corresponding continuous optimal t-th time slot, the action behavior can be obtained. In the $a(t)$ can be expressed as $a(t) = \left(p(t), q(t)\right)$, where $p(t)$ represents the set of continuous actions, and $q(t)$ represents the set of discrete actions. In order to evaluate the quality of the action in the observed state, the action value function $Q\left(s(t), a(t)\right) = Q\left(s(t), p(t), q(t)\right)$, the Bellman equation is expressed as:

$$Q\left(s(t), p(t), q(t)\right) = \underset{Re(t),s(t+1)}{\mathrm{E}}\left(Re(t) + \gamma \max_{q_{(\cdot)}(t)\in q(t)} \sup_{p_{(\cdot)}(t)\in p(t)} Q\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t)\right)\middle|a(t) = \left(p(t), q(t)\right)\right) \tag{77}$$

where γ represents the discount parameter and $p_{(\cdot)}(t)$ represents a subset of a variable in the set $p(t)$ of continuous actions, and $q_{(\cdot)}(t)$ represents a subset of a variable in the set $q(t)$ of discrete actions. In solving the $p^*_{(\cdot)}(t) = argsup_{p_{(\cdot)}(t)\in p(t)} Q\left(s(t+1), q_{(\cdot)}(t), p_{(\cdot)}(t)\right)$ including $q_{(\cdot)}(t) \in q(t)$ We

can obtain the maximum $Q\left(s(t+1), q_{(\cdot)}(t), p^*_{(\cdot)}(t)\right)$. However, taking the optimal value of $p(t)$ is computationally difficult to handle. To solve this problem, the DDPG network maps the environment state $s(t)$ to the action vector $p^*_{(\cdot)}(t)$, which can be expressed as:

$$p^*_{(\cdot)}(t) = \mu(s(t), \omega^{\mu}) = \underset{p_{(\cdot)}(t)\in p(t)}{\arg\sup}\, Q\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t)\right) \tag{78}$$

where $\mu(\cdot)$ represents the policy function for obtaining the optimal $p_{(\cdot)}(t)$ from the set of continuous action variables $p(t)$, and ω^{μ} is defined as the set of network parameters of DDPG.

In continuous motion through DDPG, Dueling DQN parameterized state - the action value function $Q\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t)\right)$ to select the optimal discrete motion $q^*_{(\cdot)}(t)$, it can be represented as:

$$\begin{cases} q^*_{(\cdot)}(t) = \underset{q(\cdot)}{\arg\sup} Q\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t)\right) \\ Q\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t)\right) = Q\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t); \omega^{Q}\right) \\ Q\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t); \omega^{Q}\right) = V(s(t); \omega^{V}) + A\left(s(t), q_{(\cdot)}(t), p_{(\cdot)}(t); \omega^{A}\right) \end{cases} \tag{79}$$

where ω^{Q} is defined as the set of network parameters of Dueling DQN, ω^{V} and ω^{A} are defined as the parameters of the value function $\mathcal{V}$ and the advantage function $\mathcal{A}$ in Dueling DQN, respectively.

The SOADRL network is updated using Temporal Difference algorithm. First, the ω^{Q} of Dueling DQN is fixed, and the ω^{μ} of DDPG is updated by minimizing the Temporal Difference loss function. Secondly, the updated ω^{μ} is fixed, and the ω^{Q} of Dueling DQN is updated, which is related to the evaluation of discrete action. The update process can be represented as:

$$\begin{cases} y_{tar} = Re + \gamma \max_{q'_{(\cdot)}(t)\in q(t)} Q\left(s'(t), q'_{(\cdot)}(t), \mu\left(s'(t), \omega^{\mu}\right); \omega^{Q}\right), \\ \ell_{Q}(\omega^{Q}) = \frac{1}{2}\left[Q\left(s(t), q_{(\cdot)}(t), \mu\left(s(t), \omega^{\mu}\right); \omega^{Q}\right) - y_{tar}\right]^2, \\ \ell_{\mu}(\omega^{\mu}) = -\sum_{q_{(\cdot)}(t)\in q(t)} Q\left(s(t), q_{(\cdot)}(t), \mu\left(s(t), \omega^{\mu}\right); \omega^{Q}\right), \\ \omega = \omega - \delta \cdot \nabla_{\omega}\ell(\omega), \end{cases} \tag{80}$$

where y_{tar} represents the expected state-action value, $s'(t)$ is defined as the next state, and ℓ_{Q} and ℓ_{μ} represent the loss functions of Dueling DQN and DDPG, respectively. ω stands for updating parameter, δ stands for learning rate, $\nabla_{\omega}\ell(\omega)$ stands for stochastic gradient method with respect to parameter ω.

SOADRL algorithm details are shown as follows: At the beginning of each time slot, state $s(t-1)$ is initialized in Step 4, which is the position

Algorithm 1 SOADRL Algorithm.

a

1: Initialize the network with weights ω^{Q} and ω^{μ}
2: Initialize the experience reply Buffer, time slot count N, UAV's remaining energy $E_{remain}(t)$
3: **for** each episode **do**
4: Observe the initial state s(t-1) ;
5: **for** t = 1:N **do**
6: Obtain continuous actions $p_{(\cdot)}(t) = \mu(s(t), \omega^{\mu})$;
7: Choose discrete action $q_{(\cdot)}(t) \in q(t)$ based on $\epsilon - greedy$ policy;
8: Execute action $a(t) = \left(p(t), q(t)\right)$;
9: Observe the $Re(t)$ and the new $s(t+1)$;
10: Save the tuple $\{s(t), a(t), Re(t), s(t+1)\}$ obtained by interacting with the environment into the experience pool;
11: Randomly select a small batch of tuples $\{s(\zeta), a(\zeta), Re(\zeta), s(\zeta+1)\}$ from the experience pool;
12: **if** The time slot ends in state $s(\zeta+1)$ **then**
13: $y(\zeta) = Re(\zeta)$;
14: **else**
15: $y(\zeta) = Re(\zeta) + \gamma \max_{q_{(\cdot)}(t)\in q(t)} Q\left(s(\zeta+1), q_{(\cdot)}(\zeta+1), \mu(s(\zeta+1), \omega^{\mu}); \omega^{Q}\right)$;
16: **end if**
17: Use data $\{y(\zeta), s(\zeta), a(\zeta+1)\}$ to compute gradient with $\nabla_{\omega^{\mu}}\ell_{\mu}(\omega^{\mu})$ and $\nabla_{\omega^{Q}}\ell_{Q}(\omega^{Q})$;
18: Update the weights by $\omega^{\mu}(t+1) \leftarrow \omega^{\mu}(t) - \delta\nabla_{\omega^{\mu}}\ell_{\mu}(\omega^{\mu})$ and $\omega^{Q}(t+1) \leftarrow \omega^{Q}(t) - \delta\nabla_{\omega^{Q}}\ell_{Q}(\omega^{Q})$;
19: $t = t + 1$;
20: Enter the new episode if the UAV's remaining energy $E_{remain}(t)$;
21: **end for**
22: **end for**
23: End

of the UAV $\varrho_j(t)$. The state $s(t)$ is then first entered into the Actor network, and the continuous action associated with all the discrete actions is obtained in step 6. Next, the state $s(t)$ and the continuous action enter the Critic network as input. In Step 7, the discrete action $q_{(\cdot)}(t)$ is selected using the $\epsilon - greedy$ policy exploration mechanism. In Step 8, the UAV performs the action function, where action $a(t)$ is the parameter. Then, in step 9, the UAV can obtain a reward function $Re(t)$ and enter the next state $s(t+1)$. In Step 10, the tuple $\{s(t), a(t), Re(t), s(t+1)\}$ will be saved in the experience pool. During the training phase, a batch of data is extracted from the experience pool in Step 11, and the expected state action values are calculated in steps 12-16. The random gradient of the loss function for the Actor network and Critic network is then calculated by means of data sampling in Step 17. When the random gradient of the loss function is obtained, the network parameters are updated in step 18. In step 20, if the

remaining energy $E_{remain}(t) \leq 0$ of the UAV is less than or equal to 0, the algorithm will enter a new time slot ahead of time. Finally, the algorithm will enter a new time slot after the current time slot ends.

4.4. Overall algorithm design

Based on latent space and SOADRL, we design Cooperative Long-term Average Cost Minimization Optimization Algorithm (CLACMO) to solve the joint optimization problem of UAV deployment and task caching in UAV-assisted MEC systems.

Algorithm 2 CLACMO Algorithm.

1: Initialize actor π_ζ with random $\zeta, \omega^Q, \omega^\mu$;
2: Initialize discrete action embedding table G_ω and conditional VAE q_ϕ, q_ψ with random parameters ω, ϕ, ψ;
3: Initialize state information $s(0)$;
4: Prepare replay buffer $\mathcal{D}$;
5: **while** not reach maximum warm-up training time **do**
6: Update ω, ϕ, ψ using samples in $\mathcal{D}$;
7: **end while**
8: **while** not reach maximum total environment steps **do**
9: Observe current system state $s(t)$; /* select latent actions by actor network */
10: $g, z = \pi_\zeta(t) + \epsilon_g$ with $\epsilon_g \sim \Gamma(0, \sigma)$; /* decode into original hybrid actions */
11: Decode $\mathrm{q}(t) = f_D(g), \mathrm{p}(t) = q_\psi(z, s(t), g)$ by decoder;
12: Execute $(p(t), q(t))$, get reward $Re(t)$ and new state $s'(t)$;
13: Store $\left(s(t), p(t), q(t), g, z, Re(t), s'(t)\right)$ in replay buffer $\mathcal{D}$; /* evaluate hybrid actions by critic network */
14: Sample a mini-batch experience from $\mathcal{D}$;
15: Update ω^Q and ω^μ according to the **Algorithm 1**;
16: π_ζ with policy gradient according to the **Algorithm 1**;
17: **while** not reach representation training times **do**
18: Update ω, ϕ, ψ using samples in $\mathcal{D}$;
19: **end while**
20: **end while**
21: End

Cooperative long-term average cost minimization optimization is proposed algorithm to solve the joint optimization problem of UAV placement and resource allocation in UAV-assisted MEC system. The details of the CLACMO presented are described in Algorithm 2. The network parameters and embedded table parameters are randomly initialized, and the system state $s(t)$ is initialized with the starting position of the UAV. There are two main phases in training: the warm-up phase and the learning phase. During the warm-up phase, the encoder and decoder are pre-trained with the experience found in the replay buffer $\mathcal{D}$ (steps 5-7). In the learning phase, the Actor outputs a latent action g, z, which is disturbed by Gaussian probe noise based on the current state $s(t)$. The decoder then decodes the latent action g, z into the original mixed action $a(t) = \left(p(t), q(t)\right)$ to interact with the environment and get a return $Re(t)$ and a new state $s'(t)$. Experience $\left(s(t), p(t), q(t), g, z, Re(t), s'(t)\right)$ will be stored in the retransmission buffer $\mathcal{D}$ (step 13). In order to avoid the correlation of input samples, this paper randomly draws a small set of experiences from $\mathcal{D}$ (Step 14). This article will calculate the loss function based on the estimate of the Critic network and then use the policy gradient to update the parameters of the Actor and Critic network (steps 15,16). In addition, the decoder and encoder are updated simultaneously during the training phase to accommodate changes in the data distribution (Steps 17-19). The framework of the CLACMO Method is shown in Fig. 2.

In the initialization stage, first, the parameters ζ of the Actor network π_ζ, the discrete - action embedding table σ_ω, and the parameters ω, ϕ, ψ of the conditional variational auto - encoders q_ϕ, q_ψ are randomly initialized. At the same time, the system state $\mathbf{s}(0)$ is initialized with the starting position of the UAVs, and the replay buffer $\mathcal{D}$ is prepared to store the experience data generated during subsequent training processes.

In the pre-heating stage, before reaching the maximum pre - heating training time, samples are obtained from the replay buffer $\mathcal{D}$, and the parameters ω, ϕ, ψ of the discrete - action embedding table and the encoder - decoder are updated through the loss function. The purpose of this stage is to pre - train the encoder and decoder to adapt to the data distribution initially, laying a foundation for the subsequent learning stage.

In the learning stage, without reaching the maximum total number of environment steps, first, action generation is carried out. The Actor network outputs latent action vectors $\mathbf{g}, \mathbf{z}$ according to the current system state $\mathbf{s}(t)$. To enhance exploration, noise $\epsilon_\mathbf{g} \sim \Gamma(0, \sigma)$ from a Gaussian distribution is added to the output vectors, that is, $\mathbf{g}, \mathbf{z} = \pi_\zeta(t) + \epsilon_\mathbf{g}$. Then, the latent action vectors are decoded into original hybrid actions by the decoder, where the discrete action $\mathbf{q}(t) = f_D(\mathbf{g})$ and the continuous action $\mathbf{p}(t) = q_\psi(\mathbf{z}, \mathbf{s}(t), \mathbf{g})$. Secondly, in the environment interaction, the decoded hybrid actions $(\mathbf{p}(t), \mathbf{q}(t))$ are executed. The UAVs interact with the environment, obtain the reward $\mathrm{Re}(t)$ and enter a new state $\mathbf{s}'(t)$. Then, the information $(\mathbf{s}(t), \mathbf{p}(t), \mathbf{q}(t), \mathbf{z}, \mathrm{Re}(t), \mathbf{s}'(t))$ generated by this interaction is stored in the replay buffer $\mathcal{D}$. For network update, to reduce the correlation of samples, a small batch of experience data is randomly sampled from the replay buffer $\mathcal{D}$. According to the SOADRL algorithm, the loss function is calculated using these samples, and the parameters of the Actor network and the Critic network are updated based on the policy gradient, specifically updating ω^Q and ω^π. In addition, before reaching the indicated number of training times, the samples in the replay buffer $\mathcal{D}$ are used again to update the parameters ω, ϕ, ψ of the discrete - action embedding table and the encoder - decoder, enabling them to adapt to the dynamic changes in the data distribution.

After training is completed, the trained model can be used in the UAV - assisted mobile edge computing system. By inputting the system state and outputting the corresponding action decisions, the joint optimization of task offloading and cache placement is achieved to reduce the long - term average weighted cost of the system.

4.5. Complexity analysis

The algorithm complexity proposed can be analyzed from two main aspects. First, there is the complexity of encoding and decoding mixed actions. Second, consider the complexity of the network of actors and critics. For a fully connected neural network with a fixed number of hidden layers and neurons, the computational complexity of the backpropagation algorithm is proportional to the product of the input size and the output size.

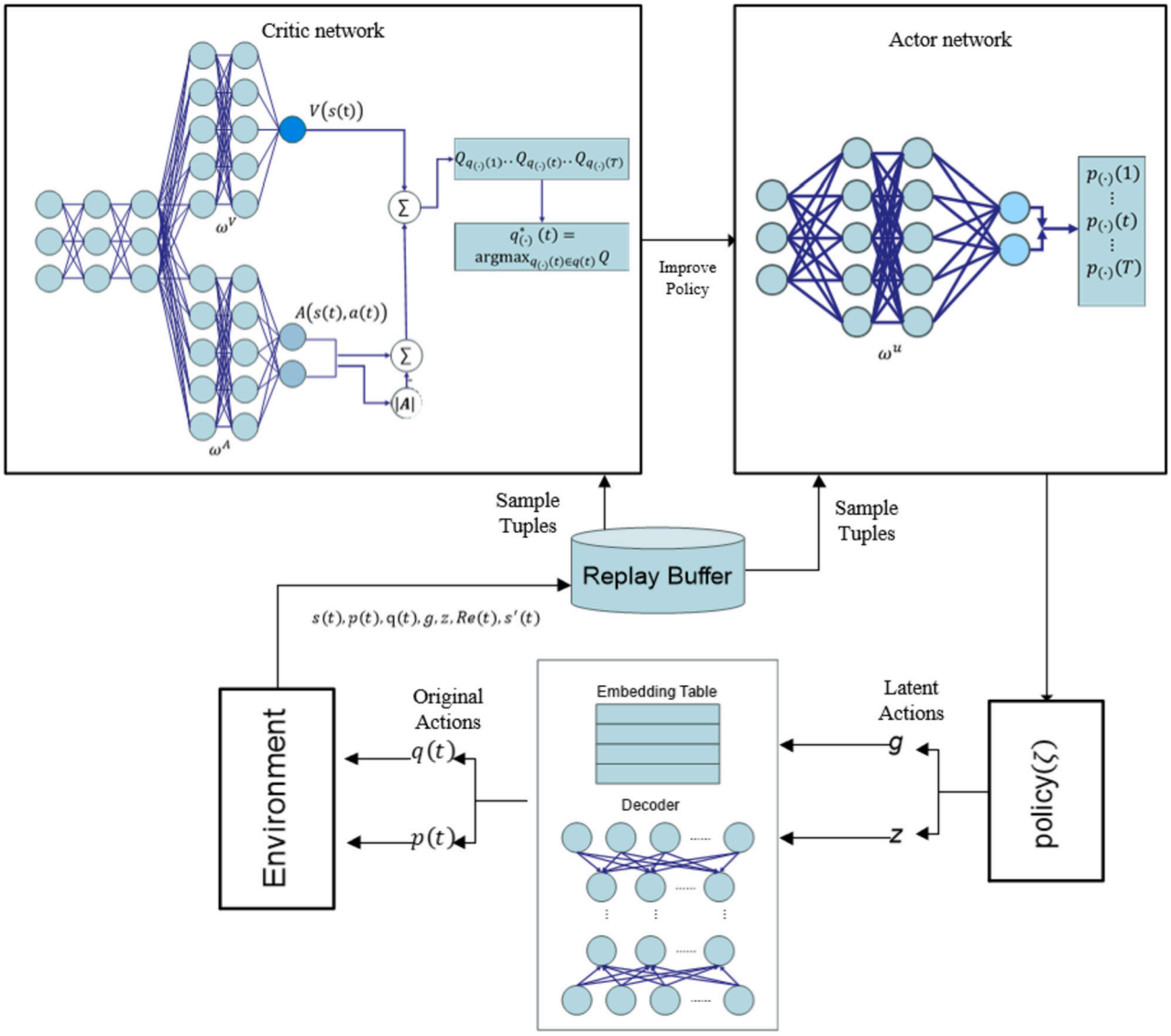


Fig. 2. CLACMO Algorithm.

For encoding and decoding of mixed actions, the encoded input size is $dim + Ml_1 + \mathcal{X}_P = 2I + 5J + J^2 + 2I(J+M) + Ml_1 + 2$, where $dim = I + 3J$ $\mathcal{X}_P = J^2 + 2J(I+1) + I(M+1) + 2$. The output size of the decoder is l_2, so the encoder complexity is $\mathcal{O}((IJ + Ml_1)l_2)$. The decoder input size for $dim + Ml_1 + l_2 = I + 3J + Ml_1 + l_2$, output size of $dim + \mathcal{X}_P = J^2 + (2I+5)J + 2I + IM + 2 + Ml_1 + l_2$. Therefore, the decoder complexity is $\mathcal{O}(I^2Jl_1 + IJl_2 + IJ^2)$.

When training the network of Actor and Critic, the input size of Actor is the system space size $dim = I + 3J$, and the output size of the dimension of the mixed action represents the space is $Il_1 + l_2$, so the complexity of Actor is $\mathcal{O}\big((Il_1 + l_2)(I+J)\big)$. The input size of Critic is $dim + \mathcal{X}_P + M$ and the output is 1, so the Critic complexity is $\mathcal{O}(IJ)$.

5. Simulation result

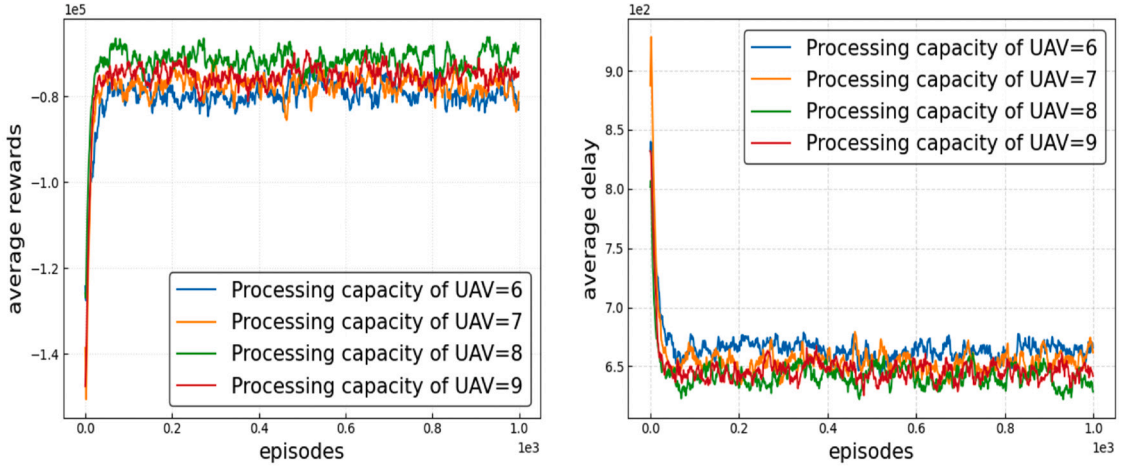
In this section, the paper describes the experimental steps and introduces alternative solutions for comparison. Then, experimental results and correlation analysis are given to verify the performance of the algorithm.

5.1. Experimental setup

The simulation for this article was performed in PyTorch. The parameters of UE tasks are set according to the data scale and computing requirements of common applications based on [40]. The parameters of UAVs take into account the chip performance, flight requirements and energy consumption comprehensively. The channel parameters are determined based on the communication frequency band allocation and data traffic. Other parameters are set based on the actual environmental noise and signal propagation characteristics. These parameters are closely combined with the actual scenarios and can effectively reflect the system performance. Consider a UAV-assisted MEC application scenario where 50 UEs are randomly distributed in a $500 \times 500m^2$ area. Several UAVs with random initial positions can help the UE to complete the calculation task. For UE tasks, the data size A_i is set from 1 to 3 MB, and the calculated workload (number of CPU cycles) A_i^{cycle} is generated randomly from [300,500] Megacycles. For UAVs, the computing power $f_i^j(t)$ will be set from 10 to 30 Gigacycles; the flight height of UAV is set to 75 m. The maximum

Table 4
Parameter Settings On Simulation.

Parameter	Symbol	Value
Total communication bandwidth	W	$80MHz$
UE's Maximum transfer rate	$p_i(t)$	$300mW$
noise power spectral density	N_0	$-150dBm/Hz$
Effective switching capacitance	κ_{UAV}	10^{-28}
Maximum cycle frequency of the local CPU	f_l^{max}	$0.5GHz$
Maximum CPU cycle frequency of the UAV	F_u	$3GHz$
The maximum frequency assigned by the cloud to the UE	$f_{Cloud-i}^{max}$	$2GHz$
Maximum flight speed of the UAV	v_{max}	$5m/s$
Path loss of LoS and NLoS	η_0, η_1	$0.1, 21$
Environmental constant A, B	A, B	$9.61; 0.16$
Actor learning rate γ_1,	γ_1	3×10^{-5}
Critic learning rate γ_2	γ_2	3×10^{-5}
Represent model learning rate γ_3	γ_3	3×10^{-5}
G2A channel bandwidth	$G2A$	$20MHz$
A2A channel bandwidth	$A2A$	$40MHz$
A2G channel bandwidth	$A2G$	$60MHz$



(a) Average reward (b) Average delay

Fig. 3. Impact of different processing capacities of UAV on reward and delay.

transmission power $p_j(t)$ is set to 5 W, the effective switching capacitor κ_{UAV} is set to 10^{-28}, and the noise power is set to -100 dBm. Path losses A, B are set to 9.61, 0.16. The channel bandwidth value of G2A, A2A and G2A communication are set to 20 MHz, 40 MHz and 10 MHz respectively. Learning rate γ_1 of Actor network, learning rate γ_2 of Critic network, representative model learning rate γ_3 are based on [23] (Table 4).

5.2. Impact of processing capacities

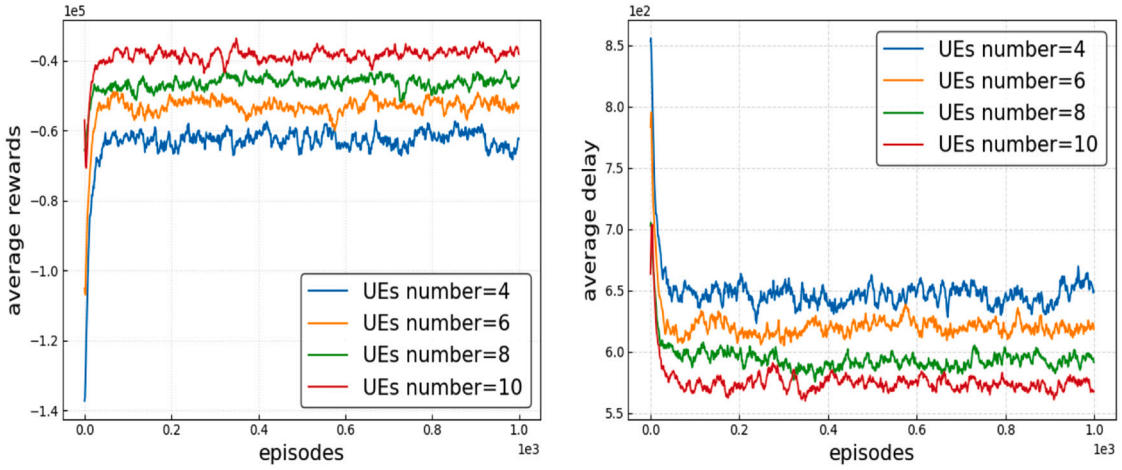
Given the different processing capabilities of the UAV, this paper first evaluates the performance of the proposed scheme in solving problem P1. In general, the average reward corresponding to the goal in the problem statement in Fig. 3 (a) tends to converge and increases with processing power. When the average reward increases as the increase of processing capacity, the reason is that the improvement in processing capacity makes the UAV process more task data per unit time. Physically, this shortens the calculation time of tasks on the UAV, enables more tasks to be completed smoothly, and makes the system resources more efficiently utilized. Therefore, the benefits obtained by the system increase, which is reflected in the improvement of the average reward.

The reward function mainly consists of two parts, the weighted average sum of total system delay and total system energy consumption. The average system delay is drawn by calculation according to Fig. 3 (b). Due to the increase of computing resources, the average delay converges quickly and is negatively correlated with the size of the processing power, that is, it decreases with the increase of the processing power. The reduction in the average delay is because the powerful processing capacity accelerates the calculation speed of tasks, reduces the waiting and processing time of tasks in the system, and thus reduces the overall average delay.

5.3. Impact of number of UEs

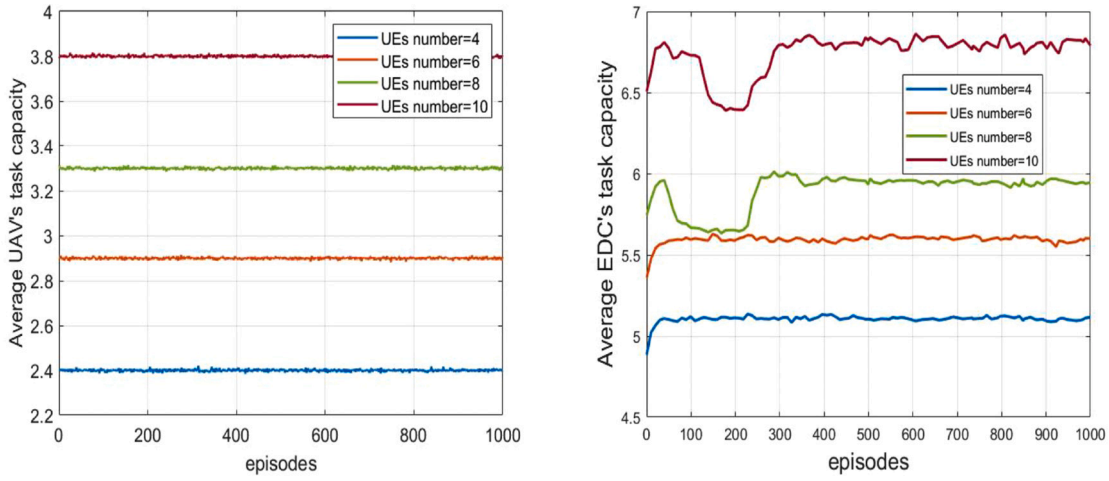
In Figs. 4 and 5, the recommendations presented are measured by the impact of the number of UE.

In Fig. 4(a), it makes sense that the average reward tends to converge as the number of UEs increases. In Fig. 4(a), the average reward tends to converge as the number of user equipment (UE) increases. The reason is that the system gradually reaches a relatively stable state when handling more UE tasks. More UEs mean more task inputs, but the system's task allocation and resource scheduling mechanisms enable the overall benefit to stabilize within a certain range. For example, as the number of UEs increases, although the task volume increases, the tasks are more evenly distributed among different processing nodes (such as UAVs, edge data centers, etc.), resulting in the stabilization of the average reward.



(a) Average reward (b) Average task delay

Fig. 4. Impact of different situation on reward and delay.



(a) Average UAV's task capacity (b) EDC's task capacity

Fig. 5. Impact of different numbers of UEs.

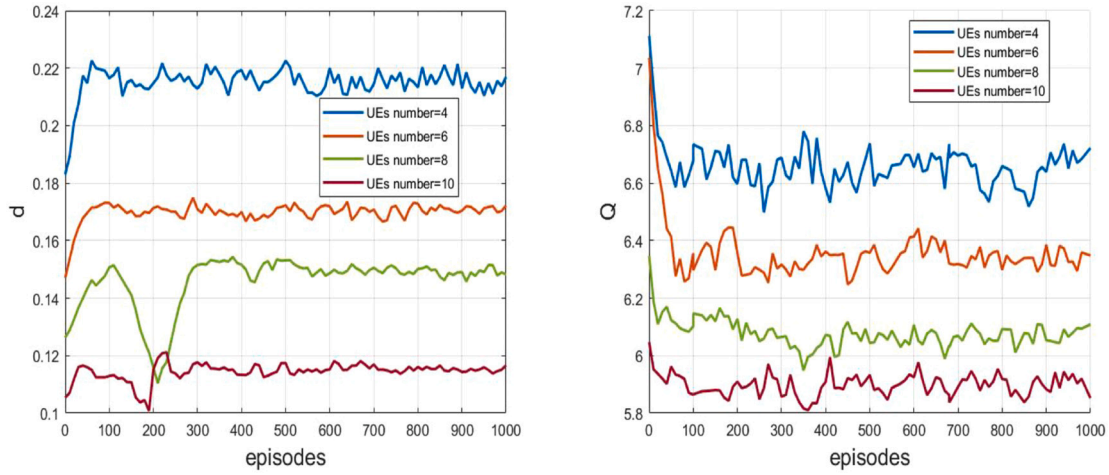
In Fig. 4(b) the latency decreases as the number of UEs increases because the more computing resources there are, the more processing power there is. The delay decreases as the number of UEs increases. This is because more UEs bring more computing resources. When the number of UEs increases, the system can perform task offloading and resource allocation more flexibly, enabling tasks to be processed faster and reducing the waiting time in the queue, thus reducing the overall delay. For example, more UEs may offload tasks to edge data centers or the cloud with stronger computing capabilities, and these nodes can process tasks quickly, thereby reducing the average delay of the system.

However, it makes sense that the backlog of UAV tasks in Fig. 5(a) would increase as the number of UEs increased. As the number of user equipment (UE) increases, the backlog of tasks on the unmanned aerial vehicle (UAV) increases. The reason is that although the overall computing resources of the system increase, the computing capacity of the UAV is relatively limited. When the number of UEs grows, the amount of tasks received by the UAV rises significantly, and its processing speed cannot keep up with the growth rate of tasks, resulting in a backlog of tasks on the UAV. Although some tasks will be offloaded to the edge data center (EDC) or the cloud, the UAV still needs to handle a certain amount of tasks. With its limited processing capacity, the backlog of tasks becomes more serious. In the simulation in this paper, the average number of tasks is similar for different UE numbers. UE slot task $D_i^l(t)$, UAV slot task $D_i^j(t)$ decreases with the increase of UE number. The average amount of UAV task processing data will increase with the increase of UE and eventually converge. These facts are illustrated in formula form in Fig. 7.

$$\mathrm{d} = \frac{\sum_{t=1}^{N} \frac{\sum_{i=1}^{I} D_i^l(t)}{I}}{N} \tag{81}$$

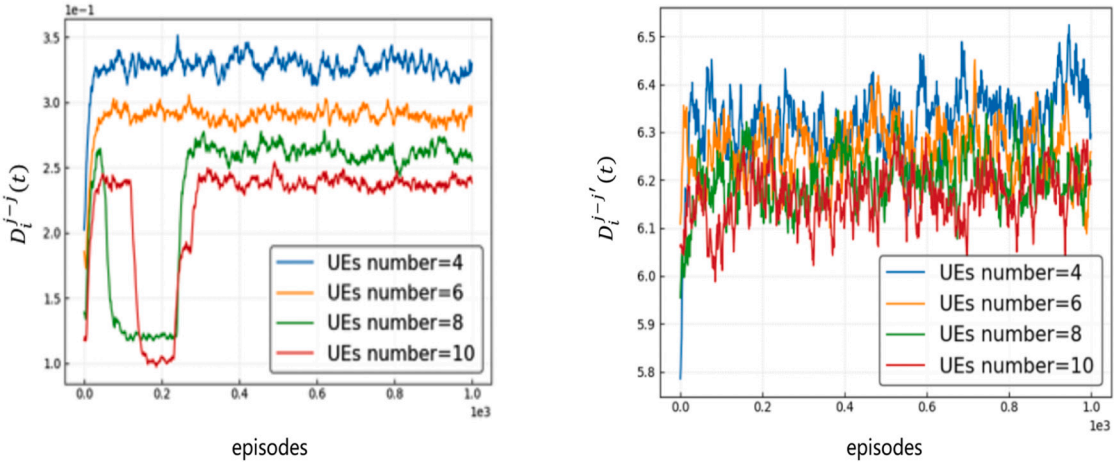
$$\mathrm{Q} = \frac{\sum_{t=1}^{N} \frac{\sum_{i=1}^{I} D_i^j(t)}{I}}{N} \tag{82}$$

Eq. (81) represents the average UE local processing task amount, and Eq. (82) represents the average UAV processing task amount. Similar results were observed for a different number of UAVs. However, the amount of task data increases as I increases, resulting in an overall increment in the number of tasks the system needs to process. The number of tasks processed by EDC is shown in Fig. 5(b), indicating that the number of tasks



(a) Average $D_i^l(t)$ (b) Average $D_i^j(t)$

Fig. 6. Impact of different numbers of UEs on $D_i^l(t)$ and $D_i^j(t)$.



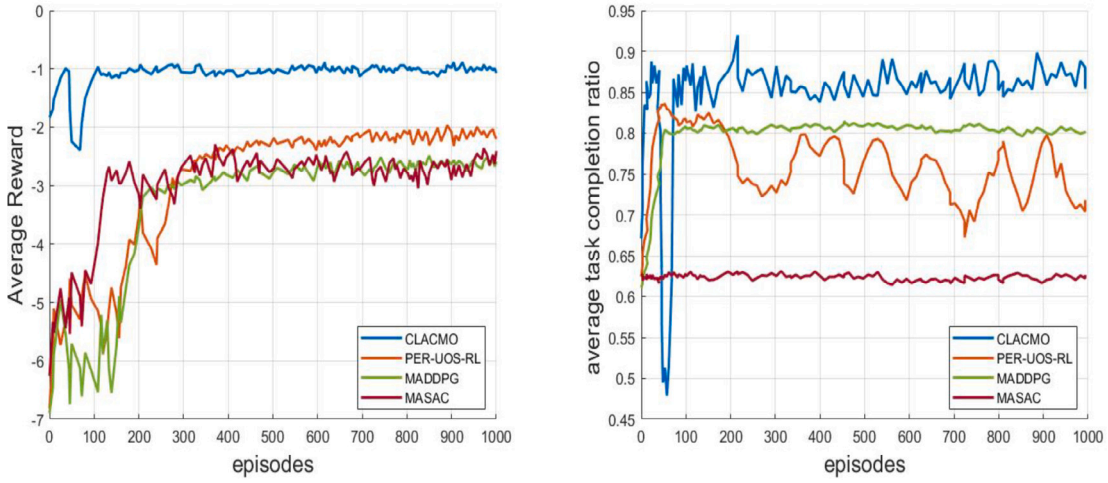
(a) Average $D_i^{j-j}(t)$ (b) Average $D_i^{j-j'}(t)$

Fig. 7. Impact of different numbers of UEs on $D_i^{j-j}(t)$ and $D_i^{j-j'}(t)$.

processed increases with the increase of the number of UE. The results were the same for different numbers of UAVs. As the number of UEs increases, the number of tasks processed by the EDC rises. This is because as the number of UEs grows, the system will allocate more tasks to the EDC with stronger computing power for processing. The increase in UE tasks makes the system readjust the task allocation strategy and is more inclined to use the resources of the EDC to meet the task processing requirements. Therefore, the number of tasks processed by the EDC shows an upward trend.

In Fig. 6, the amount of local computing tasks and UAV tasks decreases with the increase of UE number, which indicates that with the increase of UE number, more tasks will be offloaded to EDC edge server or cloud server for computing services. As the number of user equipment (UE) increases, the amount of local computing tasks decreases. This is because the system makes dynamic adjustments based on resource utilization and task processing efficiency. When the number of UEs grows, more tasks will be offloaded to edge computing nodes (such as UAVs or edge data centers) or the cloud to take advantage of their stronger computing capabilities and resources, thus reducing the computing burden on local devices and resulting in a decrease in the amount of local computing tasks. As the number of UEs increases, the amount of tasks on the UAV decreases. This may be due to the change in the task redistribution strategy of the system as the number of UEs rises. On the one hand, more tasks are directly offloaded to the edge data center or the cloud. On the other hand, even if tasks are offloaded to the UAV, the UAV may further transfer some tasks to other more suitable nodes for processing based on its own load situation and the communication and cooperation mechanism with other nodes to balance the task load of the entire system. Therefore, the amount of tasks on the UAV shows a decreasing trend.

In Fig. 7(a) of this paper, the average value of UAV time-slot task $D_i^{j-j}(t)$ decreases with the increase of UE and eventually tends to be consistent, indicating that with the increase of task data, UE tasks will be centrally unloaded to edge servers for computing services. Due to the limited service cache and computing cache resources of a single UAV, UE tasks will be centrally unloaded to edge servers for computing services. More tasks will be forwarded to other UAVs or EDCs through UAV relays for calculation, so the data volume of the final service will stabilize when the unmanned computing capacity reaches the upper limit. As the number of user equipment (UE) increases, the average value of the UAV's single-slot tasks gradually decreases and tends to stabilize. This is because as the number of UEs grows, the total number of tasks increases significantly. The system will allocate more tasks to the edge data center (EDC) or the cloud for processing. Since the service cache and computing cache resources of a single UAV are limited and cannot handle too many tasks, the amount of tasks it processes relatively decreases, resulting in a decrease in the average value and finally stabilizing when the system resource allocation reaches a new balance.



(a) Average reward (b) Average completion rate

Fig. 8. Algorithm comparisons.

In Fig. 7(b), the overall trend of UAV forwarding processing data volume is that the queue data volume decreases with the increase of UE number, indicating that with the increase of UE number, UE tasks increase dramatically, and the UAV offloads more tasks to edge data center EDC or remote cloud server due to its limited computing power. As the number of UEs increases, the queue data volume of the UAV's forwarded processing data volume decreases. As the UE tasks increase dramatically and the UAV has limited computing power, more tasks are offloaded to the edge data center EDC or the remote cloud server. As a result, the amount of task data waiting to be forwarded or processed on the UAV will decrease, causing the queue data volume to show a downward trend.

5.4. Algorithm comparisons

The CLACMO Method is compared with the following benchmark algorithms.

- PER-UOS-RL [30]: The PER-UOS-RL algorithm combines the UOS-RL algorithm based on the Multi-Agent Twin Dual-Delay Deep deterministic (MATD3) strategy gradient with a prioritized experience replay (PER) mechanism. The high-dimensional continuous action space is addressed by the UOS-RL algorithm's policy networks and evaluation networks trained through experience sharing, while the PER mechanism optimizes the training efficiency by prioritizing experience data based on their importance. Thereby, the convergence performance of the algorithm is enhanced. The PER-UOS-RL algorithm is based on jointly optimizing the UAVs' trajectories and MDs' offloading strategies to minimize the total computational cost of the system.
- MASAC [6]: The algorithm combines the Soft Actor Critic (SAC) algorithm with a centralized training and decentralized execution framework. The entropy of the strategy distribution is optimized while maximizing the reward thereby improving the stability and explorability of the algorithm. The MASAC algorithm is based on optimizing task division and power allocation strategy to maximize the total utility of the system.
- MADDPG [35]: The algorithm combines the DDPG algorithm with a centralized training decentralized execution architecture and is widely employed for multi-agent collaborative tasks.

In the performance evaluation of reinforcement - learning algorithms, the average reward is one of the key indicators for measuring the effectiveness of an algorithm. This paper compares the average - reward performance of four algorithms, namely CLACMO, PER - UOS - RL, MADDPG, and MASAC, during the training process. The relevant results are shown in the Fig. 8.

We consider the deployment of 2 UAVs and 10 randomly distributed UEs. It can be clearly observed from the Fig. 8(a) that the CLACMO demonstrates excellent performance in terms of the average reward. In the early stage of training, although the average reward of the CLACMO fluctuates to some extent, it rises rapidly and stabilizes at around -1. This indicates that the algorithm can learn quickly and converge to a good strategy to obtain a higher reward.

The average reward of the PER - UOS - RL algorithm starts at a low level and gradually increases during the training process, eventually stabilizing at approximately -2. This shows that the algorithm can continuously optimize its strategy during training, but compared with the CLACMO, the average - reward level it converges to is lower. The average reward of the MADDPG algorithm is at a low level from the start of training, rising slowly as the number of training episodes increases, and finally stabilizing at about -3. This indicates that the MADDPG algorithm faces certain challenges in the learning process, and its strategy - optimization speed is relatively slow. The average reward of the MASAC algorithm fluctuates greatly in the early stage of training and then gradually increases and stabilizes around -3. Compared with other algorithms, the instability of the MASAC algorithm in the early stage of training may be due to its environmental exploration and strategy - update mechanisms. Fig. 8(b) further verifies that the proposed scheme outperforms the other three benchmark algorithms in terms of task - completion rate performance and stability.

In summary, the CLACMO performs the best among these four algorithms and can quickly and stably obtain a higher average reward during the training process. In contrast, the PER - UOS - RL, MADDPG, and MASAC algorithms perform relatively weakly in terms of the average reward.

6. Conclusion

This paper focuses on the offloading cache placement of UE applications and tasks in a 4-layer system consisting of UAV, edge data center, and remote cloud. The main objective is to minimize the long-term system average weighted cost by jointly designing offloading decision variables,

UAV flight path, EDC cache decision variables, UAV cache decision variables, channel ratio and jointly optimizes system delay and system energy consumption. To solve this problem, we propose CLMACO algorithm based on DRL. The algorithm first converts this problem into MDP. Secondly, the latent space is used to preprocess the mixed variables, which solves the problem that the traditional deep reinforcement learning algorithm has poor training effect when facing the mixed action space. Finally, the training process is optimized based on SOADRL algorithm. A large number of simulation data show that compared with other algorithms, the proposed algorithm can significantly shorten the total system delay and reduce the system energy consumption. In the future, we will conduct real - world experiments in various scenarios, and optimize the algorithm based on results. Besides, we will study the impact of environmental factors on UAV flight energy consumption to formulate a more accurate model.

CRediT authorship contribution statement

Kai Xue: Writing – review & editing, Writing – original draft, Software, Methodology, Formal analysis. **Linbo Zhai:** Visualization. **Yumei Li:** Methodology. **Zekun Lu:** Data curation. **Wenjie Zhou:** Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] M. Asim, A.A. Abd El-Latif, M. ELAffendi, W.K. Mashwani, Energy consumption and sustainable services in intelligent reflecting surface and unmanned aerial vehicles-assisted mec system for large-scale Internet of things devices, IEEE Trans. Green Commun. Netw. 6 (2022) 1396–1407.
[2] S. Bi, L. Huang, Y.-J.A. Zhang, Joint optimization of service caching placement and computation offloading in mobile edge computing systems, IEEE Trans. Wirel. Commun. 19 (2020) 4947–4963.
[3] L. Chen, J. Xu, S. Ren, P. Zhou, Spatio–temporal edge service placement: a bandit learning approach, IEEE Trans. Wirel. Commun. 17 (2018) 8388–8401.
[4] S. Chen, X. Chen, Y. Chen, Z. Li, Distributed computation offloading based on stochastic game in multi-server mobile edge computing networks, in: 2019 IEEE International Conference on Smart Internet of Things (SmartIoT), IEEE, 2019, pp. 77–84.
[5] Z. Chen, L. Lei, X. Song, Multi-agent ddpg enpowered uav trajectory optimization for computation task offloading, in: 2022 IEEE 22nd International Conference on Communication Technology (ICCT), IEEE, 2022, pp. 608–612.
[6] Z. Cheng, M. Liwang, N. Chen, L. Huang, X. Du, M. Guizani, Deep reinforcement learning-based joint task and energy offloading in uav-aided 6g intelligent edge networks, Comput. Commun. 192 (2022) 234–244.
[7] B. Dai, J. Niu, T. Ren, Z. Hu, M. Atiquzzaman, Towards energy-efficient scheduling of uav and base station hybrid enabled mobile edge computing, IEEE Trans. Veh. Technol. 71 (1) (2022) 915–930.
[8] J. Du, F.R. Yu, G. Lu, J. Wang, J. Jiang, X. Chu, Mec-assisted immersive vr video streaming over terahertz wireless networks: a deep reinforcement learning approach, IEEE Internet Things J. 7 (2020) 9517–9529.
[9] G. Forecast, et al., Cisco visual networking index: global mobile data traffic forecast update, 2017–2022, Update 2017 (2019) 2022.
[10] X. Gao, L. Zhai, Service experience oriented cooperative computing in cache-enabled uavs assisted mec networks, IEEE Trans. Mob. Comput. (2024).
[11] X. Gao, X. Zhu, L. Zhai, Aoi-sensitive data collection in multi-uav-assisted wireless sensor networks, IEEE Trans. Wirel. Commun. 22 (2023) 5185–5197.
[12] X. Gao, X. Zhu, L. Zhai, Minimization of aerial cost and mission completion time in multi-uav-enabled iot networks, IEEE Trans. Commun. 71 (2023) 5335–5347.
[13] H. Hao, C. Xu, W. Zhang, S. Yang, G.-M. Muntean, Joint task offloading, resource allocation, and trajectory design for multi-uav cooperative edge computing with task priority, IEEE Trans. Mob. Comput. (2024).
[14] S. Jeong, O. Simeone, J. Kang, Mobile edge computing via a uav-mounted cloudlet: optimization of bit allocation and path planning, IEEE Trans. Veh. Technol. 67 (2017) 2049–2063.
[15] H. Ke, H. Wang, H. Sun, Medley deep reinforcement learning-based workload offloading and cache placement decision in uav-enabled mec networks, Complex Intell. Syst. 10 (2024) 3003–3023.
[16] P.W. Khan, K. Abbas, H. Shaiba, A. Muthanna, A. Abuarqoub, M. Khayyat, Energy efficient computation offloading mechanism in multi-server mobile edge computing—an integer linear optimization approach, Electronics 9 (2020) 1010.
[17] D.P. Kingma, Auto-encoding variational Bayes, arXiv preprint, arXiv:1312.6114, 2013.
[18] B. Li, H. Tang, Y. Zheng, J. Hao, P. Li, Z. Wang, Z. Meng, L. Wang, Hyar: addressing discrete-continuous action reinforcement learning via hybrid action representation, arXiv preprint, arXiv:2109.05490, 2021.
[19] N. Li, L. Zhai, Z. Ma, X. Zhu, Y. Li, Lyapunov-guided deep reinforcement learning for service caching and task offloading in mobile edge computing, Comput. Netw. (2024) 110593.
[20] Y. Li, X. Zhu, N. Li, L. Wang, Y. Chen, F. Yang, L. Zhai, Collaborative content caching and task offloading in multi-access edge computing, IEEE Trans. Veh. Technol. 72 (2022) 5367–5372.
[21] Z. Liao, G. Yin, X. Tang, P. Liu, A cooperative community-based framework for service caching and task offloading in multi-access edge computing, IEEE Trans. Netw. Serv. Manag. (2024).
[22] N. Lin, H. Tang, L. Zhao, S. Wan, A. Hawbani, M. Guizani, A pddqnlp algorithm for energy efficient computation offloading in uav-assisted mec, IEEE Trans. Wirel. Commun. 22 (2023) 8876–8890.
[23] B. Liu, Y. Wan, F. Zhou, Q. Wu, R.Q. Hu, Resource allocation and trajectory design for miso uav-assisted mec networks, IEEE Trans. Veh. Technol. 71 (2022) 4933–4948.
[24] S. Ma, S. Song, L. Yang, J. Zhao, F. Yang, L. Zhai, Dependent tasks offloading based on particle swarm optimization algorithm in multi-access edge computing, Appl. Soft Comput. 112 (2021) 107790.
[25] T. Ma, H. Zhou, B. Qian, N. Cheng, X. Shen, X. Chen, B. Bai, Uav-Leo integrated backbone: a ubiquitous data collection approach for b5g Internet of remote things networks, IEEE J. Sel. Areas Commun. 39 (2021) 3491–3505.
[26] G.K. Pandey, D.S. Gurjar, H.H. Nguyen, S. Yadav, Security threats and mitigation techniques in uav communications: a comprehensive survey, IEEE Access 10 (2022) 112858–112897.
[27] K. Poularakis, J. Llorca, A.M. Tulino, I. Taylor, L. Tassiulas, Joint service placement and request routing in multi-cell mobile edge computing networks, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 10–18.
[28] R. Qureshi, Ericsson mobility report. Ericsson, Stockholm, Technical Report Sweden, Tech. Rep. EAB-14, 28658, 2014.
[29] J. Ren, Y. He, G. Huang, G. Yu, Y. Cai, Z. Zhang, An edge-computing based architecture for mobile augmented reality, IEEE Netw. 33 (2019) 162–169.
[30] H. Shi, Y. Tian, H. Li, J. Huang, L. Shi, Y. Zhou, Task offloading and trajectory scheduling for uav-enabled mec networks: an madrl algorithm with prioritized experience replay, Ad Hoc Netw. 154 (2024) 103371.
[31] Q. Song, S. Jin, F.-C. Zheng, Completion time and energy consumption minimization for uav-enabled multicasting, IEEE Wirel. Commun. Lett. 8 (2019) 821–824.
[32] M.V. Tarasenkov, E.S. Poznakharev, A.V. Fedosov, Non-line-of-sight atmospheric optical communication in the visible wavelength range between uav and the ground surface, Atmosphere 15 (2023) 21.

[33] T.X. Tran, K. Chan, D. Pompili, Costa: cost-aware service caching and task offloading assignment in mobile-edge computing, in: 2019 16th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON), IEEE, 2019, pp. 1–9.
[34] K. Wang, K. Yang, C.S. Magurawalage, Joint energy minimization and resource allocation in c-ran with mobile cloud, IEEE Trans. Cloud Comput. 6 (2016) 760–770.
[35] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, L. Hanzo, Multi-agent deep reinforcement learning-based trajectory planning for multi-uav assisted mobile edge computing, IEEE Trans. Cogn. Commun. Netw. 7 (2020) 73–84.
[36] C. Xu, C. Zhan, J. Liao, J. Gong, Computation throughput maximization for uav-enabled mec with binary computation offloading, in: ICC 2022-IEEE International Conference on Communications, IEEE, 2022, pp. 4348–4353.
[37] G. Xu, N. Zhang, M. Xu, Z. Xu, Q. Zhang, Z. Song, Outage probability and average ber of uav-assisted dual-hop fso communication with amplify-and-forward relaying, IEEE Trans. Veh. Technol. 72 (2023) 8287–8302.
[38] Y. Xu, T. Zhang, J. Loo, D. Yang, L. Xiao, Completion time minimization for uav-assisted mobile-edge computing systems, IEEE Trans. Veh. Technol. 70 (2021) 12253–12259.
[39] Z. Yang, S. Bi, Y.-J.A. Zhang, Online trajectory and resource optimization for stochastic uav-enabled mec systems, IEEE Trans. Wirel. Commun. 21 (2022) 5629–5643.
[40] Y. Zeng, S. Chen, Y. Cui, J. Yang, Y. Fu, Joint resource allocation and trajectory optimization in uav-enabled wirelessly powered mec for large area, IEEE Internet Things J. 10 (2023) 15705–15722.
[41] Y. Zeng, J. Xu, R. Zhang, Rotary-wing uav enabled wireless network: trajectory design and resource allocation, in: 2018 IEEE Global Communications Conference (GLOBECOM), IEEE, 2018, pp. 1–6.
[42] L. Zhai, P. Zhao, K. Xue, Y. Li, C. Cheng, Task offloading and multi-cache placement in multi-access mobile edge computing, Comput. Netw. (2025) 111030.
[43] L. Zhai, X. Zhu, C. Cheng, Energy efficient data evacuation path for multi-uav system based on multi-objective optimization method, IEEE Trans. Veh. Technol. (2025).
[44] J. Zhang, L. Zhou, Q. Tang, E.C.-H. Ngai, X. Hu, H. Zhao, J. Wei, Stochastic computation offloading and trajectory scheduling for uav-assisted mobile edge computing, IEEE Internet Things J. 6 (2018) 3688–3699.
[45] P. Zhang, C. Wang, C. Jiang, A. Benslimane, Uav-assisted multi-access edge computing: technologies and challenges, IEEE Internet Things Mag. 4 (2021) 12–17.
[46] Z. Zhang, H. Zhou, D. Li, Joint optimization of multi-user computing offloading and service caching in mobile edge computing, in: 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS), IEEE, 2021, pp. 1–2.
[47] T. Zhao, I.-H. Hou, S. Wang, K. Chan, Red/led: an asymptotically optimal and scalable online algorithm for service caching at the edge, IEEE J. Sel. Areas Commun. 36 (2018) 1857–1870.
[48] F. Zhou, Y. Wu, R.Q. Hu, Y. Qian, Computation rate maximization in uav-enabled wireless-powered mobile-edge computing systems, IEEE J. Sel. Areas Commun. 36 (2018) 1927–1941.
[49] X. Zhu, L. Zhai, N. Li, Y. Li, F. Yang, Multi-objective deployment optimization of uavs for energy-efficient wireless coverage, IEEE Trans. Commun. (2024).